ASSIGNMENT 3 - CO3

Constructors, Destructors and Operator Overloading in C++

Name	S Mohamed Aydin
Register Number	192424090
GitHub Repository	https://github.com/mohamedaydin006/DSA0102
Submission date	07/11/2026

Every program in this report was compiled and executed; the outputs, compiler messages, sanitizer reports and timings shown are the captured results (sanitizer reports have the libc stack frames and shadow-memory dump removed for readability). Timings are best-of-N on one machine and are only meant to show trends. Predictions in Q3 and Q5 were derived by hand from the C++ rules before the programs were run.

Q	Topic
1	Constructors and constructor overloading
2	Copy constructors: shallow and deep copies
3	Dynamic constructors and destructors
4	Overloading arithmetic and stream operators
5	Overloading increment, comparison and subscript operators
6	The rules of operator overloading (Text class)
7	Complex-number class from first principles

Q1 - Constructors and Constructor Overloading

1. Implementation

`Book` has one **master constructor** (title, author, copies) and two delegating constructors: the donated-book constructor (title only) and the default constructor (empty record). Every constructor uses a member initializer list. The only place that checks the number of copies is the private function `valid()`, which the master constructor calls in its initializer list, so no Book can be created with a negative count, whichever constructor is used.

Source: Q1/q1_book.cpp

```
// Q1 - Library Book records: constructors, initializer lists, delegation
#include <iostream>
#include <string>
#include <utility>

class Book {
```

```
    std::string title;
    std::string author;
    int copies;

    // the ONLY place where the number of copies is checked
    static int valid(int c)
    {
        if (c < 0) { std::cerr << "  [warning] invalid copies " << c << " -> set to 0\n"; return 0; }
        return c;
    }
public:
    // master constructor: full details (every other constructor delegates to it)
    Book(std::string t, std::string a, int c)
        : title(std::move(t)), author(std::move(a)), copies(valid(c)) {}
    // donated book: title only
    explicit Book(const std::string& t) : Book(t, "Unknown (donated)", 1) {}
    // empty record, filled later
    Book() : Book("Untitled", "Unknown", 0) {}

    void setCopies(int c) { copies = valid(c); }
    void print(const char* label) const
    {
        std::cout << label << ": [" << title << " | " << author << " | copies=" << copies << "]\n";
    }
};

int main()
{
    Book empty;                                  // default
    Book donated("Wings of Fire");               // title only
    Book full("The C++ Programming Language", "Stroustrup", 5);
    empty.print("empty  ");
    donated.print("donated");
    full.print("full   ");

    std::cout << "--- invalid number of copies ---\n";
    Book bad("Broken Book", "Nobody", -7);       // negative copies
    bad.print("bad    ");
    Book garbage("Garbage", "Nobody", -2147483647 - 1);
    garbage.print("garbage");
    full.setCopies(-1);                          // also blocked after construction
    full.print("full   ");
}
```

Output: Q1/out_q1_book.txt

```
$ g++ -std=c++17 -Wall -Wextra q1_book.cpp -o q1_book && ./q1_book
empty  : [Untitled | Unknown | copies=0]
donated: [Wings of Fire | Unknown (donated) | copies=1]
full   : [The C++ Programming Language | Stroustrup | copies=5]
--- invalid number of copies  ---
  [warning] invalid copies -7 -> set to 0
bad    : [Broken Book | Nobody | copies=0]
  [warning] invalid copies -2147483648 -> set to 0
garbage: [Garbage | Nobody | copies=0]
  [warning] invalid copies -1 -> set to 0
full   : [The C++ Programming Language | Stroustrup | copies=0]
```

All three constructors were exercised. A negative count (-7, and the most negative int, which stands for a garbage value) is reported and replaced by 0; `setCopies(-1)` is also blocked because it uses the same `valid()` function.

2. Design and analysis

- **When does the compiler supply a default constructor?** Only when the class declares no constructor at all. As soon as any constructor is declared (for example `Book(std::string, int)`), the implicit default constructor is no longer generated; if one is still needed it must be written, or requested with `= default`. The experiment below shows the consequence for `Book shelf[3];`.
- **Why initializer lists?** The members are *initialised* directly with their final value. With assignment in the body, each member is first default-constructed and then overwritten (for `std::string` that is an extra construction and assignment). For `const` members, reference members and members without a default constructor the initializer list is the only legal way. Delegation also requires the list form: a delegating constructor may call only the target constructor in its list.
- **Initialisation order.** Members are initialised in the order they are *declared in the class*, never in the order written in the list. Writing the list in a different order does not change anything except that it misleads the reader and can read a member before it is set (see the -Wreorder experiment).
- **Constructor delegation.** `Book()` and `Book(title)` both call `Book(title, author, copies)`, so field initialisation and validation exist once; changing the validation rule means changing one line.

State of three objects after construction (checked against the printed output above)

```
Book empty;                     Book donated("Wings of Fire");     Book full("The C++ ...", "Stroustrup", 5);
+--------------------------+    +-------------------------------+  +-----------------------------------------------------+
| title  : "Untitled"      |    | title  : "Wings of Fire"      |  | title  : "The C++ Programming.."|
| author : "Unknown"       |    | author : "Unknown (donated)"  |  | author : "Stroustrup"            |
| copies : 0               |    | copies : 1                    |  | copies : 5                       |
+--------------------------+    +-------------------------------+  +-----------------------------------------------------+
 default ctor -> delegates       explicit Book(title) -> delegates   master ctor (3 args) runs valid(5)=5
 to master ctor (0 copies)       to master ctor (1 copy)
```

3. Experimental investigation

(a) Removing the default constructor. With only `Book(std::string, int)` declared, creating `Book shelf[3];` fails to compile because every array element needs a default constructor:

Compiler error

```
$ g++ -std=c++17 -Wall -Wextra q1_nodefault.cpp
q1_nodefault.cpp: In function 'int main()':
q1_nodefault.cpp:8:26: error: no matching function for call to 'Book::Book()'
    8 | int main() { Book shelf[3]; }
      |                          ^
q1_nodefault.cpp:6:5: note: candidate: 'Book::Book(std::string, int)'
    6 |     Book(std::string t, int c) : title(t), copies(c) {}
      |     ^~~~
q1_nodefault.cpp:6:5: note:   candidate expects 2 arguments, 0 provided
q1_nodefault.cpp:3:7: note: candidate: 'Book::Book(const Book&)'
    3 | class Book {
      |       ^~~~
q1_nodefault.cpp:3:7: note:   candidate expects 1 argument, 0 provided
q1_nodefault.cpp:3:7: note: candidate: 'Book::Book(Book&&)'
q1_nodefault.cpp:3:7: note:   candidate expects 1 argument, 0 provided
```

(b) Wrong initialisation order and -Wreorder. `area` is declared after `width`, but the list names `area(width * 2)` first, so `area` is computed from `width` *before* `width` has been initialised:

Source: Q1/q1_reorder.cpp

```
// Experiment: member initialised in the wrong order (-Wreorder)
#include <iostream>
class Box {
    int width;      // declared first  -> initialised first
    int area;       // declared second -> initialised second
public:
    Box(int w) : area(width * 2), width(w) {}    // list order differs from declaration order:
                                                 // area is computed from 'width' BEFORE width is set
    void print() const { std::cout << "width=" << width << " area=" << area << " (expected area " << width * 2 <<
        ")\n"; }
};
int main() { Box b(10); b.print(); }
```

Output: Q1/out_reorder.txt

```
$ g++ -std=c++17 -Wall -Wextra q1_reorder.cpp -o q1_reorder && ./q1_reorder
q1_reorder.cpp: In constructor 'Box::Box(int)':
q1_reorder.cpp:5:9: warning: 'Box::area' will be initialized after [-Wreorder]
    5 |     int area;       // declared second -> initialised second
      |         ^~~~
q1_reorder.cpp:4:9: warning:   'int Box::width' [-Wreorder]
    4 |     int width;      // declared first  -> initialised first
      |         ^~~~~
q1_reorder.cpp:7:5: warning:   when initialized here [-Wreorder]
    7 |     Box(int w) : area(width * 2), width(w) {}    // list order differs from declaration order:
      |     ^~~
width=10 area=20 (expected area 20)
```

The warning is the evidence. At run time the printed value happened to look right on every run because the stale stack slot for `width` still held 10 from the argument; reading an uninitialised int is undefined behaviour, so correct output here proves nothing. This is why the warning must not be ignored. The fix is to list members in declaration order (or compute `area` from the parameter `w`).

(c) Negative copies were tested in the main program above (-7 and INT_MIN both become 0 with a warning; no Book with an invalid count exists).

4. Design question: when should a constructor be explicit?

Mark a constructor `explicit` when it can be called with one argument and an automatic conversion from that argument type to the class would be surprising or dangerous (for example a `Book` appearing from a plain string, or

an array of N elements appearing from an int). It is the default choice for all single-argument constructors unless the conversion is a natural identity (like `std::string` from `const char*`). The program below compiles without `explicit` and fails with it:

Source: Q1/q1_explicit.cpp

```
// Experiment: explicit. Compile with and without -DUSE_EXPLICIT
#include <iostream>
#include <string>
class Book {
    std::string title;
public:
#ifdef USE_EXPLICIT
    explicit
#endif
    Book(const std::string& t) : title(t) {}
    const std::string& getTitle() const { return title; }
};
void checkOut(Book b) { std::cout << "checked out: " << b.getTitle() << '\n'; }
int main()
{
    std::string s = "Gitanjali";
    checkOut(s);                // implicit conversion string -> Book
    checkOut(Book(s));          // explicit call: always fine
}
```

Output: Q1/out_explicit.txt

```
$ g++ -std=c++17 q1_explicit.cpp    (no explicit)
checked out: Gitanjali
checked out: Gitanjali

$ g++ -std=c++17 -DUSE_EXPLICIT q1_explicit.cpp
q1_explicit.cpp: In function 'int main()':
q1_explicit.cpp:17:14: error: could not convert 's' from 'std::string' {aka 'std::__cxx11::basic_string<char>'} to
    'Book'
   17 |     checkOut(s);                // implicit conversion string -> Book
      |              ^
      |              |
      |              std::string {aka std::__cxx11::basic_string<char>}
```

`checkOut(s)` silently turns a string into a Book. With `explicit` the compiler refuses and the programmer must write `checkOut(Book(s))`. In `q1_book.cpp` the donated-book constructor is therefore marked `explicit`.

5. Alternative: one constructor with default arguments

Source: Q1/q1_alt.cpp

```
// Alternative: ONE constructor with default arguments
#include <iostream>
#include <string>
class Book {
    std::string title, author; int copies;
public:
    Book(std::string t = "Untitled", std::string a = "Unknown", int c = 0)
        : title(std::move(t)), author(std::move(a)), copies(c < 0 ? 0 : c) {}    // check in ONE place
    void print() const { std::cout << title << " | " << author << " | " << copies << '\n'; }
};
int main()
{
    Book a; Book b("Wings of Fire"); Book c("C++", "Stroustrup", 5); Book d("Bad", "X", -3);
    a.print(); b.print(); c.print(); d.print();
}
```

Output: Q1/out_alt.txt

```
$ ./q1_alt
```

Aspect	**Overloads + delegation**	**One constructor with defaults**
Object creation time	O(1): a few assignments (delegation adds one extra	O(1): same
	call, usually inlined)	
Space per object	sizeof(Book) identical (2 strings + int)	identical
Where copies are checked	One place (master constructor / valid())	One place (the only constructor)

```
Untitled | Unknown | 0
Wings of Fire | Unknown | 0
C++ | Stroustrup | 5
Bad | X | 0
```

Aspect	Overloads + delegation	One constructor with defaults
Number of functions	3 declarations	1 declaration
Different meanings per call	Yes: donated book gets copies = 1, empty record gets 0	No: every omitted argument has the same default, so Book("X") gets 0 copies, not 1
explicit	Can mark only the title-only constructor	Cannot: it is also the default constructor and the converting constructor

Evaluation. Both designs create an object in O(1) time and O(1) space and both validate in one place. The default-argument version is shorter, but it cannot give a donated book a different default (1 copy) from an empty record (0), and it cannot be made explicit without losing implicit default construction. Delegation is the better design when different situations need different defaults, as they do here.

Q2 - Copy Constructors: Shallow and Deep Copies

1. Version 1: no copy constructor (observe the problem)

Source: Q2/q2_shallow.cpp

```
// Version 1: NO copy constructor, NO copy assignment, NO user-written special members except destructor
#include <iostream>
#include <string>
class Prescription {
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
    ~Prescription() { delete[] medicines; }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol"); p.set(1, "ORS");
    Prescription q = p;           // compiler-generated MEMBERWISE copy: q.medicines == p.medicines
    q.set(1, "Cetirizine");       // changes the ORIGINAL too
    std::cout << "p: " << p.get(0) << ' ' << p.get(1) << "   q: " << q.get(0) << ' ' << q.get(1) << std::endl;
}   // q and p both delete[] the same block -> double free
```

Run without sanitizer

```
$ g++ -std=c++17 q2_shallow.cpp && ./a.out    (no sanitizer)
p: Paracetamol Cetirizine   q: Paracetamol Cetirizine
bash: line 1:   229 Segmentation fault      ./s
exit status: 139
```

Two symptoms from the problem statement appear at once: changing the copy `q` changed the original `p` (both print Cetirizine), and the program crashes when it ends (segmentation fault, exit status 139).

AddressSanitizer report, shallow version

```
$ g++ -std=c++17 -fsanitize=address -g q2_shallow.cpp && ./a.out
p: Paracetamol Cetirizine   q: Paracetamol Cetirizine
==235==ERROR: AddressSanitizer: heap-use-after-free on address 0x507000000090 at pc 0x557ca78dd2a7 bp 0x7ffcc2451100
    sp 0x7ffcc24510f0
READ of size 8 at 0x507000000090 thread T0
    #0 0x557ca78dd2a6 in Prescription::~Prescription() /home/claude/A3/Q2/q2_shallow.cpp:9
    #1 0x557ca78dcd7a in main /home/claude/A3/Q2/q2_shallow.cpp:20

0x507000000090 is located 0 bytes inside of 72-byte region [0x507000000090,0x5070000000d8)
freed by thread T0 here:
    #0 0x7f70216ff738 in operator delete[](void*, unsigned long)
        ../../../../src/libsanitizer/asan/asan_new_delete.cpp:167
    #1 0x557ca78dd364 in Prescription::~Prescription() /home/claude/A3/Q2/q2_shallow.cpp:9
    #2 0x557ca78dcd6b in main /home/claude/A3/Q2/q2_shallow.cpp:20

previously allocated by thread T0 here:
    #0 0x7f70216fe6c8 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:98
    #1 0x557ca78dd1a0 in Prescription::Prescription(int) /home/claude/A3/Q2/q2_shallow.cpp:8
    #2 0x557ca78dc727 in main /home/claude/A3/Q2/q2_shallow.cpp:15
```

ASan classifies the crash as `heap-use-after-free` inside the destructor: the block was freed by the first destructor (`q`) and the second destructor (`p`) tried to delete the same address again. For `delete[]` on a `std::string` array, the

second delete first reads the array length stored just before the block, which is already freed, so the report appears as a use-after-free rather than the words "double free" - the root cause is the same: a double delete.

2. Version 2: Complete the Code (TODO 1, 2, 3 filled in)

Source: Q2/q2_complete.cpp

```
// Q2 - Prescription: Complete the Code (TODO 1-3 filled in)
// Compile with -DNO_COPYCTOR to delete the copy constructor and watch ASan complain.
#include <iostream>
#include <string>

class Prescription {
    int n;
    std::string* medicines;    // dynamically allocated array
public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}

#ifndef NO_COPYCTOR
    // TODO 1: deep-copy constructor
    Prescription(const Prescription& o) : n(o.n), medicines(new std::string[o.n])
    {
        for (int i = 0; i < n; ++i) medicines[i] = o.medicines[i];
    }
#endif

    // TODO 2: copy assignment (self-assignment guard, build new, copy, release old)
    Prescription& operator=(const Prescription& o)
    {
        if (this == &o) return *this;
        std::string* fresh = new std::string[o.n];
        for (int i = 0; i < o.n; ++i) fresh[i] = o.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = o.n;
        return *this;
    }

    ~Prescription()
    {
        delete[] medicines;    // TODO 3
    }

    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};

int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;        // copy constructor
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;                     // copy assignment
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
              << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Output under -fsanitize=address (expected output reproduced, no sanitizer report)

```
$ g++ ... -fsanitize=address q2_complete.cpp && ./a.out
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

The output equals the expected line `Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS`: `q` and `r` are independent of `p`. Deleting the copy constructor (compile with `-DNO_COPYCTOR`) makes `Prescription q = p;` use the compiler-generated memberwise copy again (g++ also warns with -Wdeprecated-copy that this is deprecated when a user-written assignment exists):

Sanitizer error with the copy constructor deleted

```
$ g++ ... -fsanitize=address -DNO_COPYCTOR q2_complete.cpp && ./a.out
q2_complete.cpp: In function 'int main()':
q2_complete.cpp:46:22: warning: implicitly-declared 'constexpr Prescription::Prescription(const Prescription&)' is
    deprecated [-Wdeprecated-copy]
   46 |     Prescription q = p;       // copy constructor
      |                      ^
q2_complete.cpp:21:19: note: because 'Prescription' has user-provided 'Prescription& Prescription::operator=(const
    Prescription&)'
   21 |     Prescription& operator=(const Prescription& o)
      |                   ^~~~~~~~
==178==ERROR: AddressSanitizer: heap-use-after-free on address 0x507000000090 at pc 0x556781cf1a55 bp 0x7ffde167af60
    sp 0x7ffde167af50
```

```
READ of size 8 at 0x507000000090 thread T0
    #0 0x556781cf1a54 in Prescription::~Prescription() /home/claude/A3/Q2/q2_complete.cpp:34
    #1 0x556781cf1072 in main /home/claude/A3/Q2/q2_complete.cpp:53

0x507000000090 is located 0 bytes inside of 72-byte region [0x507000000090,0x5070000000d8)
freed by thread T0 here:
    #0 0x7f55712ff738 in operator delete[](void*, unsigned long)
        ../../../../src/libsanitizer/asan/asan_new_delete.cpp:167
    #1 0x556781cf1b12 in Prescription::~Prescription() /home/claude/A3/Q2/q2_complete.cpp:34
    #2 0x556781cf1063 in main /home/claude/A3/Q2/q2_complete.cpp:53

previously allocated by thread T0 here:
    #0 0x7f55712fe6c8 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:98
    #1 0x556781cf15b2 in Prescription::Prescription(int) /home/claude/A3/Q2/q2_complete.cpp:10
    #2 0x556781cf0757 in main /home/claude/A3/Q2/q2_complete.cpp:43

SUMMARY: AddressSanitizer: heap-use-after-free /home/claude/A3/Q2/q2_complete.cpp:34 in Prescription::~Prescription()
```

3. Design and analysis

```
SHALLOW COPY (compiler-generated)

   p                            q
 +------------+              +------------------+
 | n = 2      |              | n = 2      |
 | medicines |--+        +--| medicines |
 +------------+  |        |  +------------------+
                 v        v
             +--------------------------+
             | "Paracetamol" |     ONE shared block:
             | "ORS"          |     q.set(1, ..) changes p too;
             +----------------+     ~q frees it, ~p frees it again = double delete

DEEP COPY (our copy constructor)

   p                            q
 +------------+              +------------------ +
 | n = 2      |              | n = 2      |
 | medicines |--+           | medicines |--+
 +------------+  |           +------------+  |
                 v                           v
         +----------------+          +--------------------------+
         | "Paracetamol" |          | "Paracetamol" |
         | "ORS"          |          | "Cetirizine"   |
         +----------------+          +--------------------------+
          freed once by ~p            freed once by ~q
```

- **Why the shallow version crashes:** the compiler-generated copy constructor copies the member `medicines` (a pointer), not what it points to. Both objects then own the same block. Destructors run in reverse order of construction; the first `delete[]` frees the block and the second deletes it again - undefined behaviour (a crash here).
- **Rule of Three:** if a class needs a user-written destructor, copy constructor or copy assignment operator, it almost certainly needs all three. They go together because all three deal with the *ownership* of the resource: the destructor releases it, the copy constructor must create a new owned copy, and the copy assignment must replace the old owned resource with a new copy (releasing the old one). Writing only one leaves the other two doing a pointer copy.
- **Three situations in which the copy constructor is called:** (1) initialising a new object from an existing one, `Prescription q = p;` or `Prescription q(p);` (2) passing an object *by value* to a function, (3) returning an object *by value* (when the copy is not elided; since C++17 returning a temporary is guaranteed to be elided, as the measurement below shows).
- **Copy assignment** differs from the copy constructor: the target already owns an array, so it must (a) guard against `p = p`, (b) build the new array first so an exception leaves the old state intact, and (c) delete the old array.

4. Experimental investigation

Source: Q2/q2_counts.cpp

```
// Count copies: by value vs by const reference; self-assignment test
#include <iostream>
#include <string>
static int copies = 0;
class Prescription {
    int n; std::string* medicines;
public:
    explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
```

```
    Prescription(const Prescription& o) : n(o.n), medicines(new std::string[o.n])
    { ++copies; std::cout << "  [copy ctor called]\n"; for (int i = 0; i < n; ++i) medicines[i] = o.medicines[i]; }
    Prescription& operator=(const Prescription& o)
    {
        if (this == &o) { std::cout << "  [self-assignment detected, nothing done]\n"; return *this; }
        std::string* f = new std::string[o.n];
        for (int i = 0; i < o.n; ++i) f[i] = o.medicines[i];
        delete[] medicines; medicines = f; n = o.n; return *this;
    }
    ~Prescription() { delete[] medicines; }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
void byValue(Prescription x) { std::cout << "  byValue sees " << x.get(0) << '\n'; }
void byConstRef(const Prescription& x) { std::cout << "  byConstRef sees " << x.get(0) << '\n'; }
Prescription make() { Prescription t(1); t.set(0, "Zinc"); return t; }
int main()
{
    Prescription p(2); p.set(0, "Paracetamol"); p.set(1, "ORS");
    std::cout << "1) pass by value:\n";             byValue(p);
    std::cout << "2) pass by const reference:\n";  byConstRef(p);
    std::cout << "3) initialise from another object (Prescription q = p):\n"; Prescription q = p;
    std::cout << "4) return by value (C++17 guaranteed elision):\n"; Prescription z = make();
    std::cout << "copy constructor calls in total: " << copies << '\n';
    std::cout << "5) self-assignment p = p:\n";    p = p;
    std::cout << "   p still holds: " << p.get(0) << ' ' << p.get(1) << '\n';
}
```

Output: Q2/out_counts.txt

```
$ g++ ... -fsanitize=address q2_counts.cpp && ./a.out
1) pass by value:
  [copy ctor called]
  byValue sees Paracetamol
2) pass by const reference:
  byConstRef sees Paracetamol
3) initialise from another object (Prescription q = p):
  [copy ctor called]
4) return by value (C++17 guaranteed elision):
copy constructor calls in total: 2
5) self-assignment p = p:
  [self-assignment detected, nothing done]
   p still holds: Paracetamol ORS
```

Passing by value made exactly **1 copy**; passing by const reference made **0**; `Prescription q = p` made 1; returning `make()` made 0 (guaranteed elision, C++17). In total 2 copies. Self-assignment `p = p` was detected by the guard and nothing was changed (p still holds Paracetamol and ORS). Without the guard, the assignment would delete `p`'s array and then copy from the freed block.

Source: Q2/q2_bench.cpp

```
// Time to copy prescriptions of n medicines: deep copy (new[]) vs vector (Rule of Zero) vs shallow copy
#include <chrono>
#include <iostream>
#include <string>
#include <vector>
#include <cstdio>
struct Deep {
    int n; std::string* m;
    explicit Deep(int c) : n(c), m(new std::string[c]) { for (int i = 0; i < c; ++i) m[i] = "Medicine-" +
        std::to_string(i); }
    Deep(const Deep& o) : n(o.n), m(new std::string[o.n]) { for (int i = 0; i < n; ++i) m[i] = o.m[i]; }
    ~Deep() { delete[] m; }
};
struct Shallow { int n; std::string* m; };    // pointer copy only (what the compiler does)
template <class F> double best(F f, int reps)
{
    double b = 1e30;
    for (int r = 0; r < 5; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        for (int i = 0; i < reps; ++i) f();
        auto t1 = std::chrono::steady_clock::now();
        double us = std::chrono::duration<double, std::micro>(t1 - t0).count() / reps;
        if (us < b) b = us;
    }
    return b;
}
int main()
{
    std::printf("%10s | %14s | %14s | %14s\n", "n", "deep new[] (us)", "vector (us)", "shallow (us)");
    for (int n : {10, 1000, 100000}) {
        int reps = n == 10 ? 200000 : n == 1000 ? 2000 : 20;
        Deep d(n); std::vector<std::string> v; for (int i = 0; i < n; ++i) v.push_back("Medicine-" +
            std::to_string(i));
        Shallow s{n, d.m}; volatile long sink = 0;
        double a = best([&] { Deep c(d); sink += c.n; }, reps);
        double b = best([&] { std::vector<std::string> c(v); sink += (long)c.size(); }, reps);
        double c = best([&] { Shallow t = s; sink += t.n; }, reps * 10);
```

```
        std::printf("%10d | %14.3f | %14.3f | %14.4f\n", n, a, b, c);
    }
}
```

Copy time per prescription, -O2 (best of 5)

```
$ g++ -std=c++17 -O2 q2_bench.cpp && ./a.out
         n | deep new[] (us) |    vector (us) |   shallow (us)
        10 |           0.078 |          0.050 |          0.0003
      1000 |           5.931 |          2.043 |          0.0003
    100000 |         736.653 |        364.474 |          0.0004
```

n medicines	deep copy new[] (us)	vector (us)	shallow pointer copy (us)
10	0.078	0.050	0.0003
1,000	5.93	2.04	0.0003
100,000	736.7	364.5	0.0004

Deep copy time grows linearly with n (x100 data gives roughly x120 time, i.e. O(n), with cache effects), the vector version has the same growth with a smaller constant, and the shallow copy is constant O(1) - but only because it copies nothing, which is exactly why it is wrong.

5. Conclusion

The compiler-generated copy constructor is safe when every member is itself safely copyable: built-in values, `std::string`, `std::vector`, other classes obeying the Rule of Zero, or smart pointers whose sharing is intended. You must write your own when the class owns a resource through a raw pointer, file handle, socket, lock or any handle whose copy must be a new, independent resource (or must be forbidden with `= delete`).

6. Alternative: std::vector<std::string> (Rule of Zero)

```
class Prescription {
    std::vector<std::string> medicines;        // owns its memory
public:
    explicit Prescription(int count) : medicines(count) {}
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};   // no copy constructor, no assignment, no destructor: the compiler-generated ones are correct
```

Design	Copy time	Extra space	Risk
Raw new[] + Rule of Three	O(n) (measured 737 us at n = 100,000)	O(n) (new array)	Must get 3 functions right
vector (Rule of Zero)	O(n) (measured 364 us at n = 100,000)	O(n) plus 3 pointers of bookkeeping	None: copy/assign/destroy correct by construction
Shallow copy	O(1)	O(1)	Aliasing and double delete (crash)

Both correct designs copy every medicine name, so the asymptotic cost is the same; vector is simpler and was about 2x faster in the measurement. The Rule of Zero removes the possibility of the bug entirely, so it is the preferred design unless a custom allocation strategy is required.

Q3 - Dynamic Constructors and Destructors

1. Implementation

`Event` allocates a two-dimensional seat map in its constructor (an array of `rows` pointers, each pointing to its own array of `seatsPerRow` chars) and releases every allocation in its destructor, which also prints the closing summary. The copy operations are deleted because a shallow copy would cause the double delete from Q2.

Source: Q3/q3_auditorium.cpp

```
// Q3 - Auditorium seat booking: dynamic constructor / destructor
#include <iostream>
#include <string>

class Event {
    std::string name;
```

```
    int rows, seatsPerRow;
    char** seat;                  // seat[r][s] : '.' free, 'X' booked
    int booked;
public:
    Event(const std::string& n, int r, int s)
        : name(n), rows(r < 0 ? 0 : r), seatsPerRow(s < 0 ? 0 : s), seat(nullptr), booked(0)
    {
        seat = new char*[rows];                       // array of row pointers
        for (int i = 0; i < rows; ++i) {
            seat[i] = new char[seatsPerRow];          // one block per row
            for (int j = 0; j < seatsPerRow; ++j) seat[i][j] = '.';
        }
        std::cout << "  [ctor ] " << name << " (" << rows << " x " << seatsPerRow << ")\n";
    }
    Event(const Event&) = delete;                     // owns raw pointers: forbid shallow copies
    Event& operator=(const Event&) = delete;

    bool book(int r, int s)
    {
        if (r < 0 || r >= rows || s < 0 || s >= seatsPerRow || seat[r][s] == 'X') return false;
        seat[r][s] = 'X'; ++booked; return true;
    }
    bool cancel(int r, int s)
    {
        if (r < 0 || r >= rows || s < 0 || s >= seatsPerRow || seat[r][s] != 'X') return false;
        seat[r][s] = '.'; --booked; return true;
    }
    void print() const
    {
        std::cout << "  seat map of " << name << ":\n";
        for (int i = 0; i < rows; ++i) { std::cout << "     "; for (int j = 0; j < seatsPerRow; ++j) std::cout <<
            seat[i][j]; std::cout << '\n'; }
    }
    ~Event()
    {
        for (int i = 0; i < rows; ++i) delete[] seat[i];    // release every row
        delete[] seat;                                      // then the row-pointer array
        std::cout << "  [dtor ] " << name << ": closing summary - " << booked << " of "
                  << rows * seatsPerRow << " seats booked\n";
    }
};

int main()
{
    std::cout << "1) stack object\n";
    Event a("Concert", 3, 5);
    std::cout << "2) object inside a block\n";
    {
        Event b("Seminar", 2, 4);
        b.book(0, 1); b.book(1, 3);
        std::cout << "  leaving block...\n";
    }
    std::cout << "3) heap object (new / delete)\n";
    Event* c = new Event("Drama", 2, 3);
    c->book(1, 2);
    delete c;
    std::cout << "4) booking and cancelling on 'Concert'\n";
    std::cout << "  book(0,0)=" << a.book(0, 0) << " book(0,0) again=" << a.book(0, 0)
              << " book(2,4)=" << a.book(2, 4) << " book(5,5)=" << a.book(5, 5) << '\n';
    std::cout << "  cancel(0,0)=" << a.cancel(0, 0) << " cancel(0,0) again=" << a.cancel(0, 0) << '\n';
    a.book(1, 1);
    a.print();
    std::cout << "5) zero rows\n";
    Event z("Empty hall", 0, 10);
    std::cout << "  book(0,0)=" << z.book(0, 0) << '\n';
    std::cout << "end of main\n";
}
```

Output under -fsanitize=address (no report: no leaks, no errors)

```
$ g++ -std=c++17 -Wall -Wextra -fsanitize=address -g q3_auditorium.cpp && ./a.out
1) stack object
  [ctor ] Concert (3 x 5)
2) object inside a block
  [ctor ] Seminar (2 x 4)
  leaving block...
  [dtor ] Seminar: closing summary - 2 of 8 seats booked
3) heap object (new / delete)
  [ctor ] Drama (2 x 3)
  [dtor ] Drama: closing summary - 1 of 6 seats booked
4) booking and cancelling on 'Concert'
  book(0,0)=1 book(0,0) again=0 book(2,4)=1 book(5,5)=0
  cancel(0,0)=1 cancel(0,0) again=0
  seat map of Concert:
    .....
    .X...
    ....X
5) zero rows
  [ctor ] Empty hall (0 x 10)
  book(0,0)=0
```

```
end of main
  [dtor ] Empty hall: closing summary - 0 of 0 seats booked
  [dtor ] Concert: closing summary - 2 of 15 seats booked
```

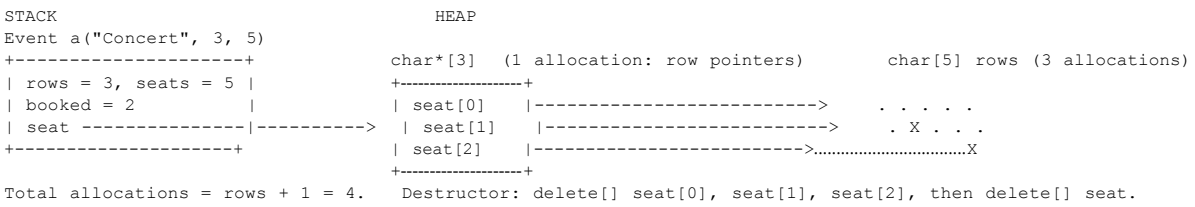
2. Order of construction and destruction (recorded above)

Kind of object	Constructed	Destroyed
Local (stack) `Concert`, `Empty hall`	when execution reaches its declaration	at the closing brace of main, in REVERSE order: Empty hall first, then Concert
Block-scope `Seminar`	when execution reaches its declaration inside the block	at the end of the block (printed before "3) heap object")
Heap `Drama` (new)	when new executes	only when delete executes; if delete is never called the destructor never runs

3. Design and analysis

- **If delete is forgotten:** the destructor never runs, so neither the closing summary is printed nor is any memory released: the whole seat map leaks for the life of the program. ASan (LeakSanitizer) report below.
- **delete instead of delete[]:** undefined behaviour. For an array of objects with destructors only the first destructor may run, and the pointer given to the allocator is not the pointer that was allocated (the array has a hidden header). ASan reports the bad free below; g++ also warns with -Wmismatched-new-delete.
- **Why a destructor takes no arguments and cannot be overloaded:** it is called implicitly (end of scope, delete, end of the program) at points where the caller has no opportunity to supply arguments, and there must be exactly one unambiguous way to clean up an object. A class therefore has exactly one destructor with the signature `~Event()`; it also has no return type.

Memory layout of the seat map (3 x 5, state after the bookings in the test)



4. Experimental investigation

delete forgotten (-DLEAK): LeakSanitizer report

```
$ g++ ... -fsanitize=address -g -DLEAK q3_leak.cpp && ./a.out

==258==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 32 byte(s) in 1 object(s) allocated from:
    #0 0x7f5586afe548 in operator new(unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:95
    #1 0x5608fa080483 in main /home/claude/A3/Q3/q3_leak.cpp:7

SUMMARY: AddressSanitizer: 32 byte(s) leaked in 1 allocation(s).
```

delete instead of delete[] (-DMISMATCH): AddressSanitizer report

```
$ g++ ... -fsanitize=address -g -DMISMATCH q3_leak.cpp && ./a.out
q3_leak.cpp: In function 'int main()':
q3_leak.cpp:11:12: warning: 'void operator delete(void*, std::size_t)' called on pointer returned from a mismatched
    allocation function [-Wmismatched-new-delete]
   11 |     delete arr;                      // should be delete[]
      |            ^~~
q3_leak.cpp:10:31: note: returned from 'void* operator new [](std::size_t)'
   10 |     Logger* arr = new Logger[3];
      |                               ^
==266==ERROR: AddressSanitizer: attempting free on address which was not malloc()-ed: 0x50b000000048 in thread T0
    #0 0x7fc226aff5e8 in operator delete(void*, unsigned long)
        ../../../../src/libsanitizer/asan/asan_new_delete.cpp:164
    #1 0x555c192eb541 in main /home/claude/A3/Q3/q3_leak.cpp:11

0x50b000000048 is located 8 bytes inside of 104-byte region [0x50b000000040,0x50b0000000a8)
allocated by thread T0 here:
    #0 0x7fc226afe6c8 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:98
    #1 0x555c192eb4c7 in main /home/claude/A3/Q3/q3_leak.cpp:10
```

Correct code: delete / delete[] (no report)

```
$ g++ ... -fsanitize=address -g q3_leak.cpp  (correct delete / delete[])
exit status 0 (no report)
```

Zero rows: `Event z("Empty hall", 0, 10)` works: `new char*[0]` is legal and returns a valid pointer to an empty array, the allocation loop and the free loop execute zero times, every `book()` is rejected by the range check and the destructor reports 0 of 0 seats.

Memory for 10 x 20, 100 x 200 and 1,000 x 2,000 maps (bytes requested from `operator new`, counted by a replacement allocator; the real heap use is slightly higher because malloc adds bookkeeping per block):

Source: Q3/q3_alt.cpp

```
// Alternative seat maps: (1) array of row pointers  (2) one flat block  (3) vector<vector<char>>
// Counts allocations by replacing global operator new.
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <new>
#include <utility>
#include <vector>
static long allocCount = 0, allocBytes = 0;
void* operator new(std::size_t n) { ++allocCount; allocBytes += (long)n; if (void* p = std::malloc(n)) return p;
    throw std::bad_alloc(); }
void* operator new[](std::size_t n) { ++allocCount; allocBytes += (long)n; if (void* p = std::malloc(n)) return p;
    throw std::bad_alloc(); }
void operator delete(void* p) noexcept { std::free(p); }
void operator delete[](void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }
void operator delete[](void* p, std::size_t) noexcept { std::free(p); }

struct Rows { char** s; int R, C;
    Rows(int r, int c) : s(new char*[r]), R(r), C(c) { for (int i = 0; i < r; ++i) { s[i] = new char[c]; for (int j =
        0; j < c; ++j) s[i][j] = '.'; } }
    ~Rows() { for (int i = 0; i < R; ++i) delete[] s[i]; delete[] s; }
    char& at(int r, int c) { return s[r][c]; } };
struct Flat { char* s; int R, C;
    Flat(int r, int c) : s(new char[(std::size_t)r * c]), R(r), C(c) { for (long i = 0; i < (long)r * c; ++i) s[i] =
        '.'; }
    ~Flat() { delete[] s; }
    char& at(int r, int c) { return s[(std::size_t)r * C + c]; } };
struct Vec { std::vector<std::vector<char>> s;
    Vec(int r, int c) : s(r, std::vector<char>(c, '.')) {}
    char& at(int r, int c) { return s[r][c]; } };

template <class M> void run(const char* name, int R, int C)
{
    allocCount = allocBytes = 0;
    auto t0 = std::chrono::steady_clock::now();
    M m(R, C);
    long a = allocCount, b = allocBytes;
    auto t1 = std::chrono::steady_clock::now();
    long cnt = 0;
    for (int rep = 0; rep < 5; ++rep) for (int r = 0; r < R; ++r) for (int c = 0; c < C; ++c) { m.at(r, c) = 'X'; cnt
        += m.at(r, c) == 'X'; }
    auto t2 = std::chrono::steady_clock::now();
    std::printf("%-22s %4d x %-4d allocs=%-5ld bytes=%-9ld build %8.3f ms | 5 sweeps %8.3f ms (%ld)\n",
        name, R, C, a, b,
        std::chrono::duration<double, std::milli>(t1 - t0).count(),
        std::chrono::duration<double, std::milli>(t2 - t1).count(), cnt);
}
int main()
{
    for (auto d : {std::pair<int,int>{10, 20}, {100, 200}, {1000, 2000}}) {
        run<Rows>("array of row pointers", d.first, d.second);
        run<Flat>("flat block r*S+s", d.first, d.second);
        run<Vec >("vector<vector<char>>", d.first, d.second);
    }
}
```

Measurements (-O2); timings for the small maps are below clock resolution and not meaningful

```
$ g++ -std=c++17 -O2 q3_alt.cpp && ./a.out
array of row pointers     10 x 20   allocs=11    bytes=280       build    0.002 ms | 5 sweeps    0.000 ms (1000)
flat block r*S+s          10 x 20   allocs=1     bytes=200       build    0.000 ms | 5 sweeps    0.001 ms (1000)
vector<vector<char>>      10 x 20   allocs=12    bytes=460       build    0.000 ms | 5 sweeps    0.001 ms (1000)
array of row pointers    100 x 200  allocs=101   bytes=20800     build    0.007 ms | 5 sweeps    0.001 ms (100000)
flat block r*S+s         100 x 200  allocs=1     bytes=20000     build    0.000 ms | 5 sweeps    0.001 ms (100000)
vector<vector<char>>     100 x 200  allocs=102   bytes=22600     build    0.002 ms | 5 sweeps    0.055 ms (100000)
array of row pointers   1000 x 2000 allocs=1001  bytes=2008000   build    0.463 ms | 5 sweeps    0.293 ms (10000000)
flat block r*S+s        1000 x 2000 allocs=1     bytes=2000000   build    0.650 ms | 5 sweeps    0.329 ms (10000000)
vector<vector<char>>    1000 x 2000 allocs=1002  bytes=2026000   build    0.368 ms | 5 sweeps    5.481 ms (10000000)
```

Map	Data bytes	Row pointers (8 B x rows)	Total requested	Allocations
10 x 20	200	80	280	11
100 x 200	20,000	800	20,800	101
1,000 x 2,000	2,000,000	8,000	2,008,000	1,001

5. Conclusion: RAII

When the release of memory is tied to the destructor, the compiler guarantees it runs on every path out of the scope - normal exit, early return, or exception - for stack and block objects. The programmer no longer has to remember a matching cleanup call at every exit, so leaks, forgotten releases and double releases (which this assignment produced deliberately) become structurally impossible for objects that are not shared. For heap objects it still depends on `delete`, which is why modern code wraps them in `std::unique_ptr`.

6. Predict the Output

Source: Q3/q3_predict.cpp

```
#include <iostream>
#include <string>
struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};
void use(Logger) { std::cout << "use "; }
int main(){
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    std::cout << "end ";
}
```

My predicted output (written before running):

```
+A +B -B copyAc use -Ac +P -P end -A
```

Actual output

```
$ ./q3_predict
+A +B -B copyAc use -Ac +P -P end -A
```

No difference between prediction and actual output. Explanation, in the order the text appears: `Logger a("A")` prints `+A`. The inner block creates `b` (`+B`) and destroys it at the closing brace (`-B`). `use(a)` takes its parameter *by value*, so the copy constructor builds the parameter from `a` with the name "A" + "c" = "Ac" (`copyAc`); the body prints `use` ; the parameter (a separate object) is destroyed when the call finishes (`-Ac`). `new Logger("P")` prints `+P` and `delete p` prints `-P`. `end` is printed, and then `a` is destroyed at the end of main (`-A`). Every created object has exactly one destruction line: A, B, Ac, P.

7. Alternative: one flat block or vector<vector<char>>

```
Flat block:  char* seat = new char[rows * seats];    element (r, s) is seat[r * seats + s]
             one allocation, one delete[], rows contiguous in memory, no row-pointer array
```

Design	Access time	Allocations	Extra space beyond the rows*seats chars	Measured at 1000 x 2000
Array of row pointers	O(1): two loads	rows + 1	rows x 8 B pointers	1,001 allocs, 8,000 B extra
Flat block	O(1): one multiply-add + load	1	none	1 alloc, 0 B extra
vector<vector<char>>	O(1): two loads	rows + 1 (+1 temporary prototype row)	rows x 24 B vector objects (+ capacity slack)	1,002 allocs, 26,000 B extra

All three give O(1) access and O(rows x seats) total space. The flat block makes one allocation instead of rows + 1 (a single `delete[]`, better locality, no pointer overhead) but the programmer must do the index arithmetic; the vector versions remove the manual `delete[]` (Rule of Zero) at the price of 24 bytes per row. In this measurement the nested vector sweep was also slower (5.5 ms vs 0.3 ms at -O2 for 5 sweeps of 2 million seats); treat that figure as machine-dependent, but the allocation counts and byte counts are exact.

Q4 - Overloading Arithmetic and Stream Operators

1. Complete the Code (TODO 1 and 2) and the carry test

Source: Q4/q4_complete.cpp

```
// Q4 - Complete the Code (TODO 1 and 2) + extra carry test
#include <iomanip>
#include <iostream>

class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: + as a member returning a NEW Money
    Money operator+(const Money& o) const
    {
        Money r;
        r.paise = paise + o.paise;
        return r;
    }

    // TODO 2: == as a member, << as a friend printing "Rs 200.00"
    bool operator==(const Money& o) const { return paise == o.paise; }

    friend std::ostream& operator<<(std::ostream& os, const Money& m)
    {
        long long v = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) os << '-';
        return os << "Rs " << v / 100 << '.' << std::setw(2) << std::setfill('0') << v % 100
                  << std::setfill(' ');
    }
};

int main()
{
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';

    // extra test: paise carry into rupees
    Money x(0, 75), y(0, 50);
    std::cout << x << " + " << y << " = " << x + y << "  (== Rs 1.25 ? " << (x + y == Money(1, 25)) << ")\n";
}
```

Output: Q4/out_complete.txt

```
$ g++ -std=c++17 -Wall -Wextra q4_complete.cpp && ./a.out
Rs 200.00
true
Rs 0.75 + Rs 0.50 = Rs 1.25  (== Rs 1.25 ? true)
```

Expected output `Rs 200.00` / `true` is reproduced (120.75 + 79.25 = 200.00), and the added test confirms that 75 paise + 50 paise carries into rupees: **Rs 1.25**. The class stores paise in a `long long`, so there is no rounding error.

2. Full Money class and the bill calculation

Source: Q4/q4_money.cpp

```
// Q4 - full Money class: + - += -= == != < <= > >= << >>, scalar *, bill calculation
#include <iomanip>
#include <iostream>
#include <string>
#include <cstdlib>

class Money {
    long long paise;                        // exact integer arithmetic: no floating point error
    explicit Money(long long p, bool) : paise(p) {}
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    static Money fromPaise(long long p) { return Money(p, true); }
    long long totalPaise() const { return paise; }
```

```
    Money& operator+=(const Money& o) { paise += o.paise; return *this; }    // modifies *this, returns reference
    Money& operator-=(const Money& o) { paise -= o.paise; return *this; }
    Money& operator*=(long long k)     { paise *= k;      return *this; }   // Money * number only (Rs x Rs makes no
        sense)

    friend Money operator+(Money a, const Money& b) { a += b; return a; }  // returns a NEW object
    friend Money operator-(Money a, const Money& b) { a -= b; return a; }
    friend Money operator*(Money a, long long k)     { a *= k; return a; }
    friend Money operator*(long long k, Money a)     { a *= k; return a; }
    friend bool operator==(const Money& a, const Money& b) { return a.paise == b.paise; }
    friend bool operator!=(const Money& a, const Money& b) { return !(a == b); }
    friend bool operator< (const Money& a, const Money& b) { return a.paise <  b.paise; }
    friend bool operator> (const Money& a, const Money& b) { return b < a; }
    friend bool operator<=(const Money& a, const Money& b) { return !(b < a); }
    friend bool operator>=(const Money& a, const Money& b) { return !(a < b); }

    friend std::ostream& operator<<(std::ostream& os, const Money& m)
    {
        long long v = std::llabs(m.paise);
        if (m.paise < 0) os << "-";
        os << "Rs " << v / 100 << '.' << std::setw(2) << std::setfill('0') << v % 100;
        return os << std::setfill(' ');
    }
    friend std::istream& operator>>(std::istream& is, Money& m)   // reads like 120.75 or 80
    {
        std::string s; if (!(is >> s)) return is;
        bool neg = !s.empty() && s[0] == '-'; if (neg) s.erase(0, 1);
        std::size_t dot = s.find('.');
        long long r = std::atoll(s.substr(0, dot).c_str()), p = 0;
        if (dot != std::string::npos) { std::string f = (s.substr(dot + 1) + "00").substr(0, 2); p =
            std::atoi(f.c_str()); }
        m.paise = (r * 100 + p) * (neg ? -1 : 1);
        return is;
    }
};

int main()
{
    std::cout << "--- paise carry ---\n";
    Money a(0, 75), b(0, 50);
    std::cout << a << " + " << b << " = " << a + b << '\n';

    std::cout << "--- bill calculation with operators ---\n";
    Money rice(120, 50), oil(210, 75), soap(35, 25);
    Money total = 2 * rice + oil + 3 * soap;
    Money discount(50);
    Money payable = total - discount;
    Money paid(500);
    Money change = paid - payable;
    std::cout << "items   : " << total << "\ndiscount: " << discount << "\npayable : " << payable
              << "\npaid    : " << paid << "\nchange  : " << change << '\n';
    total += Money(0, 99);
    std::cout << "after += Rs 0.99 : " << total << '\n';

    std::cout << "--- subtraction giving a negative amount ---\n";
    Money due = Money(10) - Money(10, 25);
    std::cout << "Rs 10.00 - Rs 10.25 = " << due << "  (negative = customer still owes)\n";
    std::cout << "due < Money() ? " << std::boolalpha << (due < Money()) << '\n';

    std::cout << "--- comparisons ---\n";
    std::cout << (Money(5, 50) < Money(5, 75)) << ' ' << (Money(5, 50) == Money(5, 50)) << ' ' << (Money(5) >=
        Money(6)) << '\n';

    std::cout << "--- >> reading ---\n";
    Money r1, r2; std::cin >> r1 >> r2;
    std::cout << "read " << r1 << " and " << r2 << ", sum " << r1 + r2 << '\n';

}
```

Output: Q4/out_money.txt

```
$ printf "120.75 79.5\n" | ./q4_money
--- paise carry ---
Rs 0.75 + Rs 0.50 = Rs 1.25
--- bill calculation with operators ---
items   : Rs 557.50
discount: Rs 50.00
payable : Rs 507.50
paid    : Rs 500.00
change  : -Rs 7.50
after += Rs 0.99 : Rs 558.49
--- subtraction giving a negative amount ---
Rs 10.00 - Rs 10.25 = -Rs 0.25  (negative = customer still owes)
due < Money() ? true
--- comparisons ---
true true false
--- >> reading ---
read Rs 120.75 and Rs 79.50, sum Rs 200.25
```

Bill check by hand: 2 x 120.50 = 241.00; + 210.75 = 451.75; + 3 x 35.25 (105.75) = **557.50**; minus 50.00 discount = **507.50** payable. Paying Rs 500 leaves **-Rs 7.50** (the customer still owes 7.50). `>>` reads "120.75" and "79.5" correctly (79.5 is read as 79.50), sum Rs 200.25.

3. Design and analysis

- **Why + returns a new object but += returns *this by reference:** `a + b` must not change `a` or `b`, so it builds and returns a new value. `a += b` modifies `a` itself and returns a reference to it so that it can be chained (`(a += b) += c`) without a copy. Implementing `+` through `+=` (as in my class: `Money operator+(Money a, const Money& b){ a += b; return a; }`) keeps the arithmetic in one place.
- **Members vs friends:** compound assignment operators (`+=`, `-=`, `*=`) modify the left operand, so they are members. Symmetric binary operators (`+`, `-`, `==`, `<`, ...) are non-member friends so that conversions apply to *both* operands (see the `100 + m` experiment below). `<<` and `>>` are friends because they need the private `paise` and their left operand is a stream. In the Complete-the-Code program, as the assignment asks, `+` and `==` are members and `<<` is a friend.
- **Why << must be a non-member:** for a member function the left operand is the object, so `m << std::cout` would be required. The left operand of `std::cout << m` is `std::ostream`, a library class that cannot be modified, so the operator has to be a free function `operator<<(std::ostream&, const Money&)`; it returns the stream reference so that calls chain.
- **How a + b is translated:** as a member, `a + b` becomes `a.operator+(b)`; as a non-member it becomes `operator+(a, b)`. `std::cout << c << '\n'` becomes `operator<<(operator<<(std::cout, c), '\n')`.

4. Experimental investigation

Negative subtraction: Rs 10.00 - Rs 10.25 gives **-Rs 0.25** (see output above). Decision: Money may be negative, because a bill must be able to represent "amount still due" or a refund; the operator `<<` prints the sign once and `<` `Money()` tests for it. The alternative (throwing an exception on negative results) would make legitimate change calculation awkward. If the business rule is that a *price* may never be negative, check that in the code that creates prices, not in `operator-`.

Readability with and without operators:

Source: Q4/q4_readability.cpp

```
// Same bill, WITHOUT operator overloading (named functions) - readability comparison
#include <iostream>
struct Money { long long paise; };
Money add(Money a, Money b)       { return {a.paise + b.paise}; }
Money sub(Money a, Money b)       { return {a.paise - b.paise}; }
Money times(long long k, Money a)  { return {k * a.paise}; }
void print(Money m) { std::cout << "Rs " << m.paise / 100 << '.' << (m.paise % 100 < 10 ? "0" : "") << m.paise % 100;
    }
int main()
{
    Money rice{12050}, oil{21075}, soap{3525}, discount{5000}, paid{50000};
    Money payable = sub(add(add(times(2, rice), oil), times(3, soap)), discount);    // hard to read
    Money change  = sub(paid, payable);
    std::cout << "payable : "; print(payable); std::cout << "\nchange  : "; print(change); std::cout << '\n';
}
```

Output: Q4/out_readability.txt

```
$ ./q4_readability
payable : Rs 507.50
change  : Rs -7.0-50
```

Same calculation

```
with operators    :  Money payable = 2 * rice + oil + 3 * soap - discount;
without operators:  Money payable = sub(add(add(times(2, rice), oil), times(3, soap)), discount);
```

The operator version reads like the bill. The named-function version must be read inside-out. It also demonstrates a second advantage: my hand-written `print()` printed the negative change as `Rs -7.0-50` (it does not handle the sign), whereas the single overloaded `operator<<` handles the sign and zero-padding once for the whole program.

Double vs integer paise for 1,000,000 additions of Rs 0.10:

Source: Q4/q4_double.cpp

```
// 1,000,000 additions of Rs 0.10: double vs Money (long long paise)
#include <cstdio>
#include <chrono>
int main()
{
    const int N = 1000000;
    double d = 0.0; long long paise = 0;
    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) d += 0.10;
    auto t1 = std::chrono::steady_clock::now();
    volatile long long vp = 0;
    for (int i = 0; i < N; ++i) vp = vp + 10;
    paise = vp;
    auto t2 = std::chrono::steady_clock::now();
    std::printf("double sum  : %.10f    (expected 100000.0000000000)\n", d);
    std::printf("error       : %.10f rupees\n", d - 100000.0);
    std::printf("double == 100000.0 ? %s\n", d == 100000.0 ? "true" : "false");
    std::printf("Money sum   : Rs %lld.%02lld  (exact)\n", paise / 100, paise % 100);
    std::printf("time double : %.3f ms, time paise : %.3f ms\n",
        std::chrono::duration<double, std::milli>(t1 - t0).count(), std::chrono::duration<double, std::milli>(t2 -
            t1).count());
}
```

Output: Q4/out_double.txt

```
$ g++ -O2 q4_double.cpp && ./a.out
double sum  : 100000.0000013329    (expected 100000.0000000000)
error       : 0.0000013329 rupees
double == 100000.0 ? false
Money sum   : Rs 100000.00  (exact)
time double : 0.537 ms, time paise : 0.257 ms
```

The double total is 100000.0000013329: an error of about 1.3 millionths of a rupee, and `d == 100000.0` is **false**, because 0.10 has no exact binary representation. The paise version is exact. (The timing difference in the output is not meaningful: the paise loop uses a volatile variable.) For money, exactness matters more than speed.

5. Engineering decision: which operators make sense for money?

Operation	Allow?	Reason
Money + Money, Money - Money, +=, -=	Yes	Amounts of the same unit add and subtract
Money * integer / integer * Money	Yes	Quantity x price (implemented)
Money * Money	**No**	Rs x Rs is rupees squared, which has no meaning in billing
Money / integer	With care	Splitting a bill needs a rounding rule for leftover paise
Money / Money	Maybe	A dimensionless ratio (double), e.g. discount percentage; not a Money
==, !=, <, >, <=, >=	Yes	Exact on integers
Money + plain number	Debatable	Allowed here because the constructor is not explicit (100 means Rs 100); making the constructor explicit would remove this risk

6. Alternative: non-member friend functions (or named functions)

Source: Q4/q4_friend_vs_member.cpp

```
// 100 + m : member operator+ vs non-member (friend) operator+
#include <iostream>
class Money {
    long long paise;
public:
    Money(long long r = 0, int p = 0) : paise(r * 100 + p) {}       // NOT explicit: 100 converts to Money
#ifdef MEMBER
    Money operator+(const Money& o) const { Money r; r.paise = paise + o.paise; return r; }
#else
    friend Money operator+(const Money& a, const Money& b) { Money r; r.paise = a.paise + b.paise; return r; }
#endif
    long long get() const { return paise; }
};
int main()
{
    Money m(5, 50);
    Money x = m + 100;       // member: OK (100 converts for the right operand)
    Money y = 100 + m;       // member: ERROR, left operand is never converted
```

```
    std::cout << x.get() << ' ' << y.get() << '\n';
}
```

Output: Q4/out_friend_member.txt

```
$ g++ -std=c++17 q4_friend_vs_member.cpp    (friend version)
10550 10550

$ g++ -std=c++17 -DMEMBER q4_friend_vs_member.cpp    (member version)
q4_friend_vs_member.cpp: In function 'int main()':
q4_friend_vs_member.cpp:18:19: error: no match for 'operator+' (operand types are 'int' and 'Money')
   18 |     Money y = 100 + m;      // member: ERROR, left operand is never converted
      |               ~~~ ^ ~
      |               |     |
      |               int   Money
```

Form	m + 100	100 + m	Time / space
Member operator+	compiles (100 converts to Money as the argument)	**error**: no match for operator+(int, Money); the left operand of a member operator is never converted	O(1) / O(1)
Non-member friend operator+	compiles	compiles (both operands may convert)	O(1) / O(1)
Named functions add(a,b), equals(a,b)	compiles with explicit conversion	same	O(1) / O(1); unreadable bill code

Time and space are unchanged (an addition of two long long values, one temporary), so the choice is about correctness and usability. Only the non-member form allows `100 + m` as well as `m + 100`, which is why the full class uses friends for the symmetric operators.

Q5 - Overloading Increment, Comparison and Subscript Operators

1. Implementation

Source: Q5/q5_playlist.cpp

```
// Q5 - Time (++ prefix/postfix, ==, <) and Playlist (operator[] const / non-const with range check)
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

class Time {
    int h, m, s;
public:
    Time(int hh = 0, int mm = 0, int ss = 0)        // normalises, so Time(0,0,60) == Time(0,1,0)
    {
        int total = hh * 3600 + mm * 60 + ss;
        h = total / 3600; m = total / 60 % 60; s = total % 60;
    }
    Time& operator++()           // prefix: advance, return the NEW value by reference
    {
        if (++s == 60) { s = 0; if (++m == 60) { m = 0; ++h; } }
        return *this;
    }
    Time operator++(int)         // postfix (dummy int): remember OLD value, advance, return the COPY
    {
        Time old = *this;
        ++(*this);
        return old;
    }
    friend bool operator==(const Time& a, const Time& b) { return a.h == b.h && a.m == b.m && a.s == b.s; }
    friend bool operator< (const Time& a, const Time& b) { return a.toSeconds() < b.toSeconds(); }
    int toSeconds() const { return h * 3600 + m * 60 + s; }
    friend std::ostream& operator<<(std::ostream& os, const Time& t)
    {
        return os << t.h << ':' << std::setfill('0') << std::setw(2) << t.m << ':' << std::setw(2) << t.s <<
            std::setfill(' ');
    }
};

struct Track { std::string title; int seconds; };

class Playlist {
    std::vector<Track> tracks;
    Track invalid{"<invalid index>", 0};
public:
    void add(const std::string& t, int sec) { tracks.push_back({t, sec}); }
```

```
    int size() const { return (int)tracks.size(); }
    Track& operator[](int i)                    // non-const: caller may modify the track
    {
        if (i < 0 || i >= size()) { std::cerr << "  [error] playlist index " << i << " out of range 0.." << size() -
            1 << '\n'; return invalid; }
        return tracks[i];
    }
    const Track& operator[](int i) const        // const: used on const Playlist objects, read only
    {
        if (i < 0 || i >= size()) { std::cerr << "  [error] playlist index " << i << " out of range 0.." << size() -
            1 << '\n'; return invalid; }
        return tracks[i];
    }
};

void showFirst(const Playlist& p) { std::cout << "  const view of track 0: " << p[0].title << '\n'; }     // needs
    const operator[]

int main()
{
    std::cout << "--- values after a = t++ and b = ++t ---\n";
    Time t(0, 0, 10);
    Time a = t++;       // a gets OLD value, t becomes 0:00:11
    std::cout << "a = " << a << "  t = " << t << '\n';
    Time b = ++t;       // t becomes 0:00:12 first, b gets NEW value
    std::cout << "b = " << b << "  t = " << t << '\n';

    std::cout << "--- roll-over ---\n";
    Time r(0, 59, 59); ++r; std::cout << "0:59:59 + 1 s = " << r << '\n';
    Time r2(0, 0, 59); r2++; std::cout << "0:00:59 + 1 s = " << r2 << '\n';
    Time r3(23, 59, 59); ++r3; std::cout << "23:59:59 + 1 s = " << r3 << "  (no day wrap implemented)\n";
    std::cout << "--- comparisons ---\n";
    std::cout << std::boolalpha << (Time(0, 1, 0) == Time(0, 0, 60)) << ' ' << (Time(0, 0, 59) < Time(0, 1, 0)) << '
        ' << (Time(1, 0, 0) < Time(0, 59, 59)) << '\n';

    std::cout << "--- playback simulation ---\n";
    Playlist pl; pl.add("Intro", 3); pl.add("Raga", 5); pl.add("Finale", 4);
    Time now;
    for (int i = 0; i < pl.size(); ++i) {
        Time end = now; for (int k = 0; k < pl[i].seconds; ++k) ++end;
        std::cout << "  " << std::setw(7) << std::left << pl[i].title << std::right << " " << now << " -> " << end <<
            '\n';
        now = end;
    }
    std::cout << "  total played: " << now << '\n';
    pl[1].title = "Raga (edited)";                      // non-const [] allows modification
    std::cout << "  edited track 1: " << pl[1].title << '\n';
    showFirst(pl);
    std::cout << "--- range check ---\n";
    std::cout << "  playlist[-1]    -> " << pl[-1].title << '\n';
    std::cout << "  playlist[size] -> " << pl[pl.size()].title << '\n';
}
```

Output: Q5/out_playlist.txt

```
$ ./q5_playlist
--- values after a = t++ and b = ++t ---
a = 0:00:10  t = 0:00:11
b = 0:00:12  t = 0:00:12
--- roll-over ---
0:59:59 + 1 s = 1:00:00
0:00:59 + 1 s = 0:01:00
23:59:59 + 1 s = 24:00:00  (no day wrap implemented)
--- comparisons ---
true true false
--- playback simulation ---
  Intro   0:00:00 -> 0:00:03
  Raga    0:00:03 -> 0:00:08
  Finale  0:00:08 -> 0:00:12
  total played: 0:00:12
  edited track 1: Raga (edited)
  const view of track 0: Intro
--- range check ---
  playlist[-1]    ->   [error] playlist index -1 out of range 0..2
<invalid index>
  playlist[size] ->   [error] playlist index 3 out of range 0..2
<invalid index>
```

2. Design and analysis

- **Prefix vs postfix:** both are named `operator++`; the postfix form has an extra unnamed `int` parameter that is never used. `++t` calls `t.operator++()`, `t++` calls `t.operator++(0)` (the compiler passes the dummy 0).
- **Why postfix returns a copy and is slower:** `t++` must yield the value *before* the increment, but the object has to be incremented, so the old value is saved in a temporary (`Time old = *this;`) and returned by value after the increment. Prefix just increments and returns a reference, with no temporary.

- **Why [] needs a const and a non-const version:** on a non-const playlist, `playlist[i].title = "x"` must modify the track, so the operator returns `Track&`. On a `const Playlist&` (like `showFirst`) only const member functions may be called, so a `const Track&` version is needed; without it, const playlists could not be indexed at all.
- **Out-of-range index:** it must never read memory outside the vector. My version prints an error and returns a shared "invalid" sentinel track so the caller does not crash. In library code an exception (`at()`) is usually better because the caller can recover and a sentinel can be modified by mistake; `assert` suits debug-only checks.

3. Experimental investigation

a = t++ and b = ++t with t = 0:00:10: `a` is **0:00:10** (old value) and t becomes 0:00:11; then `b` is **0:00:12** (new value) and t is 0:00:12. **Roll-over:** 0:59:59 + 1 s = **1:00:00**; 0:00:59 + 1 s = 0:01:00; 23:59:59 + 1 s = 24:00:00 (a day wrap is not part of the requirement). **Comparisons:** `Time(0,1,0) == Time(0,0,60)` is true because the constructor normalises seconds, `0:00:59 < 0:01:00` is true, `1:00:00 < 0:59:59` is false. **Range check:** `playlist[-1]` and `playlist[size]` (index 3) both printed the error message and returned the sentinel instead of crashing.

Source: Q5/q5_bench.cpp

```
// 10,000,000 prefix vs postfix increments on a class type with a non-trivial body, plus cost of range check
#include <chrono>
#include <cstdio>
#include <vector>
struct Time { int h = 0, m = 0, s = 0;
    Time& operator++() { if (++s == 60) { s = 0; if (++m == 60) { m = 0; ++h; } } return *this; }
    Time operator++(int) { Time old = *this; ++(*this); return old; } };
struct Big { long pad[64]; int s = 0;       // a larger object makes the postfix copy visible
    Big& operator++() { ++s; pad[s & 63] += s; return *this; }
    Big operator++(int) { Big old = *this; ++(*this); return old; } };
template <class F> double best(F f) { double b = 1e30; for (int r = 0; r < 5; ++r) { auto t0 =
    std::chrono::steady_clock::now(); f(); auto t1 = std::chrono::steady_clock::now(); double ms =
    std::chrono::duration<double, std::milli>(t1 - t0).count(); if (ms < b) b = ms; } return b; }
struct Checked { std::vector<int> v; int size() const { return (int)v.size(); }
    int& operator[](int i) { if (i < 0 || i >= size()) { static int d; return d; } return v[i]; } };
int main()
{
    const long N = 10000000; volatile long sink = 0;
    double a = best([&] { Time t; for (long i = 0; i < N; ++i) ++t; sink += t.s + t.m; });
    double b = best([&] { Time t; for (long i = 0; i < N; ++i) { Time o = t++; sink += o.s; } });
    double c = best([&] { Big t; for (long i = 0; i < N; ++i) ++t; sink += t.s; });
    double d = best([&] { Big t; for (long i = 0; i < N; ++i) { Big o = t++; sink += o.s; } });
    std::printf("Time  prefix  : %8.2f ms\nTime  postfix : %8.2f ms\n", a, b);
    std::printf("Big   prefix  : %8.2f ms\nBig   postfix : %8.2f ms\n", c, d);
    Checked ch; ch.v.assign(1024, 1); int* raw = ch.v.data(); long s1 = 0, s2 = 0;
    double e = best([&] { long s = 0; for (long i = 0; i < N; ++i) s += ch[(int)(i & 1023)]; s1 = s; });
    double f = best([&] { long s = 0; for (long i = 0; i < N; ++i) s += raw[i & 1023]; s2 = s; });
    std::printf("checked [] x 10M : %8.2f ms\nunchecked     x 10M : %8.2f ms  (sums %ld %ld)\n", e, f, s1, s2);
}
```

Timings for 10,000,000 operations (best of 5)

```
$ g++ -std=c++17 -O0 q5_bench.cpp && ./a.out
Time  prefix  :   12.55 ms
Time  postfix  :   34.55 ms
Big   prefix  :   16.96 ms
Big   postfix :   162.84 ms
checked [] x 10M :    37.64 ms
unchecked    x 10M :    16.76 ms  (sums 10000000 10000000)

$ g++ -std=c++17 -O2 q5_bench.cpp && ./a.out
Time  prefix  :    4.29 ms
Time  postfix  :    6.55 ms
Big   prefix  :    0.00 ms
Big   postfix :   19.19 ms
checked [] x 10M :     2.59 ms
unchecked    x 10M :     2.60 ms  (sums 10000000 10000000)
```

Test (10,000,000 operations)	-O0	-O2
Time prefix ++	12.6 ms	4.3 ms
Time postfix ++	34.6 ms (2.8x)	6.6 ms (1.5x)
Large object (520 B) prefix	17.0 ms	0.00 ms - loop optimised away, not meaningful
Large object postfix	162.8 ms (9.6x)	19.2 ms
checked operator[]	37.6 ms	2.59 ms
unchecked raw access	16.8 ms	2.60 ms

Postfix is slower in every measurable case, and the gap grows with the size of the object because every `t++` copies the whole object. At -O2 the compiler removes much of the copy for a tiny class like `Time` (1.5x remaining) but not for a large one. The range check costs about 2 ns per access at -O0 (37.6 vs 16.8 ms per 10 million) and nothing measurable at -O2 (2.59 vs 2.60 ms) because the branch is perfectly predictable.

4. Conclusion

Prefer `++i` for class types when the old value is not needed: it never creates a temporary, so it is never slower than `i++`, and for objects with non-trivial copy cost (iterators of containers, big numbers) it can be far faster. For built-in ints the compiler removes the difference, but the habit costs nothing.

5. Predict the Output

Source: Q5/q5_predict.cpp

```
#include <iostream>
struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; } // prefix
    Counter operator++(int) { Counter old = *this; ++v; return old; } // postfix
};
int main()
{
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

My predicted output (written before running): `5 7 7`

Actual output

```
$ ./q5_predict
5 7 7
```

No difference. `c++` returns the old copy (v = 5) and then increments c to 6, so `a.v` = 5. `++c` increments c to 7 and returns a reference to c itself; `b` is initialised from it, so `b.v` = 7, and finally `c.v` = 7. `a` and `b` differ because postfix returns the value *before* the increment, prefix the value *after*. The **postfix** version (`operator++(int)`) makes the copy (`Counter old = *this;`), and returns it by value.

6. Alternative: at(i) that throws, or unchecked operator[]

```
Track& at(int i) { if (i < 0 || i >= size()) throw std::out_of_range("playlist index"); return tracks[i]; }
Track& operator[](int i) { return tracks[i]; }        // unchecked, like a built-in array: undefined behaviour if i is
    bad
```

Variant	Time per access	Behaviour on bad index	Measured (10M accesses)
operator[] + range check, prints error	O(1) + 1 comparison	message + sentinel, program continues	37.6 ms (-O0) / 2.59 ms (-O2)
at(i), throws std::out_of_range	O(1) + 1 comparison	exception, caller can catch it	same cost on the good path
Unchecked operator[]	O(1)	undefined behaviour (crash or silent corruption)	16.8 ms (-O0) / 2.60 ms (-O2)

All three are O(1) per access; the check adds one comparison (or two, for both bounds). The measured cost is about 2 ns per access at -O0 and unmeasurable at -O2, so the check is worth its price almost always. Use the unchecked form only after profiling shows the check matters.

Q6 - The Rules of Operator Overloading

1. Implementation

`Text` owns a dynamically allocated char array. It has a constructor from `const char*`, a deep-copy copy constructor, a copy assignment with a self-assignment guard, a destructor, `+=` and `+` (concatenation), `==`/`!=` and `<<`. Static counters measure copy-constructor calls, allocations and characters copied.

Source: Q6/q6_text.cpp

```
// Q6 - Text class for an SMS gateway: Rule of Three + operator overloading
#include <cstring>
#include <iostream>
#include <string>
#include <utility>

class Text {
    char* data;
    std::size_t len;
public:
    static inline long copies = 0, allocs = 0, charsCopied = 0;       // instrumentation

    Text(const char* s = "") : data(nullptr), len(std::strlen(s))
    {
        data = new char[len + 1]; ++allocs;
        std::memcpy(data, s, len + 1); charsCopied += (long)len;
    }
    Text(const Text& o) : data(new char[o.len + 1]), len(o.len)         // deep copy
    {
        ++copies; ++allocs; std::memcpy(data, o.data, len + 1); charsCopied += (long)len;
    }
    Text& operator=(const Text& o)                                      // member: needs *this, must return *this
    {
        if (this == &o) return *this;                                   // self-assignment guard
        char* fresh = new char[o.len + 1]; ++allocs;
        std::memcpy(fresh, o.data, o.len + 1); charsCopied += (long)o.len;
        delete[] data;
        data = fresh; len = o.len;
        return *this;
    }
    ~Text() { delete[] data; }

    std::size_t length() const { return len; }
    const char* c_str() const { return data; }

    Text& operator+=(const Text& o)
    {
        char* fresh = new char[len + o.len + 1]; ++allocs;
        std::memcpy(fresh, data, len); std::memcpy(fresh + len, o.data, o.len + 1);
        charsCopied += (long)(len + o.len);
        delete[] data; data = fresh; len += o.len;
        return *this;
    }
    friend Text operator+(const Text& a, const Text& b)                 // new object, builds directly (no extra copy)
    {
        Text r; delete[] r.data; --Text::allocs;
        r.len = a.len + b.len; r.data = new char[r.len + 1]; ++allocs;
        std::memcpy(r.data, a.data, a.len); std::memcpy(r.data + a.len, b.data, b.len + 1);
        charsCopied += (long)r.len;
        return r;
    }
    friend bool operator==(const Text& a, const Text& b) { return a.len == b.len && std::memcmp(a.data, b.data,
        a.len) == 0; }
    friend bool operator!=(const Text& a, const Text& b) { return !(a == b); }
    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return os << t.data; }
    static void reset() { copies = allocs = charsCopied = 0; }
};

int main()
{
    Text a("Hello, "), b("SMS "), c("gateway");
    std::cout << "a + b + c = " << (a + b + c) << '\n';

    Text::reset();
    Text r = a + b + c;
    std::cout << "evaluating a + b + c: copy-ctor calls=" << Text::copies << ", allocations=" << Text::allocs
              << ", chars copied=" << Text::charsCopied << '\n';

    std::cout << "--- comparison ---\n";
    std::cout << std::boolalpha << (Text("abc") == Text("abc")) << ' ' << (Text("abc") != Text("abd")) << ' ' <<
        (Text("") == Text()) << '\n';

    std::cout << "--- empty and long messages ---\n";
    Text e;
    std::cout << "empty length=" << e.length() << " [" << e << "]\n";
    Text big(std::string(1000, 'x').c_str());
```

```
    Text big2 = big + big;
    std::cout << "long message length=" << big.length() << ", concatenated length=" << big2.length() << '\n';
    std::cout << "empty + a = [" << (e + a) << "]  a + empty = [" << (a + e) << "]\n";

    std::cout << "--- self assignment ---\n";
    Text s("keep me");
    Text& ref = s;
    s = ref;
    std::cout << "after s = s : " << s << '\n';
    s = s = s;
    std::cout << "after s = s = s : " << s << '\n';
}
```

Output under -fsanitize=address (no report)

```
$ g++ -std=c++17 -Wall -Wextra -fsanitize=address -g q6_text.cpp && ./a.out
a + b + c = Hello, SMS gateway
evaluating a + b + c: copy-ctor calls=0, allocations=2, chars copied=29
--- comparison ---
true true true
--- empty and long messages ---
empty length=0 []
long message length=1000, concatenated length=2000
empty + a = [Hello, ]  a + empty = [Hello, ]
--- self assignment ---
after s = s : keep me
after s = s = s : keep me
```

Tests passed: concatenation, equality and inequality, the empty message (length 0, `empty + a` and `a + empty` give a), a 1000-character message and its 2000-character concatenation, and self-assignment (`s = s`, and `s = s = s`) which leaves the text unchanged.

2. Design and analysis

Operator	Why it cannot be overloaded
`::` scope resolution	Its left operand is a type or namespace name, not an expression; there is no object to pass to a function
`.` member access	Overloading it would change how every member access works inside the class itself; the language needs it to be fixed (`->` can be overloaded for smart pointers)
`.*` pointer-to-member	Same reason as `.`: it is a fixed member-access mechanism
`?:` conditional	It evaluates only one of two branches (short-circuit), which a function call with both arguments evaluated cannot imitate
`sizeof`	Resolved at compile time from the type; a function cannot return a different size than the compiler uses

- **The rules:** the *precedence* and *associativity* of an operator do not change (`a + b * c` still multiplies first); the *arity* (number of operands) does not change (a unary `!` cannot become binary); you cannot invent new operators (such as `**`); and at least one operand must be a user-defined type, so `int + int` can never be redefined. Default arguments are not allowed for operator functions (except `()`).
- **Why operator= must be a member function:** the language requires it. The compiler generates a copy assignment for every class and a free `operator=` would conflict with it; a member also guarantees that the left operand is a modifiable object of the class (no implicit conversion can change the target). The same holds for `[]`, `()` and `->`.
- **When an operator should be a friend:** when it needs private data and cannot be a member, mainly because the left operand is not the class (`<<`, `>>`) or because both operands should be treated symmetrically (`+`, `==` so that `"abc" + text` works through conversion). If the public interface is enough, a plain non-friend free function is better (less coupling).

3. Experimental investigation

Copies made by a + b + c (7 + 4 + 7 characters): copy-constructor calls = **0**, allocations = **2** (one per `+`), characters copied = **29** = (7 + 4) + (7 + 4 + 7). `operator+` builds its result directly and returns it, and C++17 guarantees there is no copy for the returned temporary; the first temporary `a + b` is bound to a const reference for the second `+`.

Address sanitizer. My first draft of the copy constructor allocated `len` bytes and copied `len + 1` (forgetting the terminating zero). ASan found it immediately:

Source: Q6/q6_bug.cpp

```
// BUGGY first draft of Text (for the ASan exercise): copy constructor forgets the '\0' byte
#include <cstring>
#include <iostream>
class Text {
    char* data; std::size_t len;
public:
    Text(const char* s = "") : len(std::strlen(s)) { data = new char[len + 1]; std::memcpy(data, s, len + 1); }
    Text(const Text& o) : len(o.len) { data = new char[len]; std::memcpy(data, o.data, len + 1); }    // BUG: should
        be len + 1
    ~Text() { delete[] data; }
    const char* c_str() const { return data; }
};
int main() { Text a("hello"); Text b = a; std::cout << b.c_str() << '\n'; }
```

ASan on the buggy draft

```
$ g++ ... -fsanitize=address -g q6_bug.cpp && ./a.out  (BUGGY draft)
==451==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000035 at pc 0x7f5937efb303 bp 0x7fff9d3523f0
    sp 0x7fff9d351b98
WRITE of size 6 at 0x502000000035 thread T0
    #0 0x7f5937efb302 in memcpy
        ../../../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_memintrinsics.inc:115
    #1 0x55e6dde3071d in Text::Text(Text const&) /home/claude/A3/Q6/q6_bug.cpp:8
    #2 0x55e6dde303ea in main /home/claude/A3/Q6/q6_bug.cpp:12

0x502000000035 is located 0 bytes after 5-byte region [0x502000000030,0x502000000035)
allocated by thread T0 here:
    #0 0x7f5937efe6c8 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:98
    #1 0x55e6dde30687 in Text::Text(Text const&) /home/claude/A3/Q6/q6_bug.cpp:8
    #2 0x55e6dde303ea in main /home/claude/A3/Q6/q6_bug.cpp:12
```

The report says WRITE of size 6 at 0 bytes after a 5-byte region: the allocation has to be `len + 1`. In `q6_text.cpp` (the final class) all allocations are `len + 1` and the ASan run is clean.

100,000 concatenations, Text vs std::string (building a string of 100,000 characters one character at a time):

Source: Q6/q6_bench.cpp

```
// 100,000 concatenations: Text vs std::string, three styles
#include <chrono>
#include <cstring>
#include <cstdio>
#include <string>
#include <utility>

class Text {                        // (A) operator+ builds a result directly
    char* d; std::size_t n;
public:
    Text(const char* s = "") : n(std::strlen(s)) { d = new char[n + 1]; std::memcpy(d, s, n + 1); }
    Text(const Text& o) : n(o.n) { d = new char[n + 1]; std::memcpy(d, o.d, n + 1); }
    Text(Text&& o) noexcept : d(o.d), n(o.n) { o.d = nullptr; o.n = 0; }
    Text& operator=(Text o) noexcept { std::swap(d, o.d); std::swap(n, o.n); return *this; }
    ~Text() { delete[] d; }
    std::size_t size() const { return n; }
    Text& operator+=(const Text& o) { char* f = new char[n + o.n + 1]; std::memcpy(f, d, n); std::memcpy(f + n, o.d,
        o.n + 1); delete[] d; d = f; n += o.n; return *this; }
    friend Text operator+(Text a, const Text& b) { a += b; return a; }          // (B) += on a copy (by-value
        parameter)
};
template <class F> double best(F f) { double b = 1e30; for (int r = 0; r < 3; ++r) { auto t0 =
    std::chrono::steady_clock::now(); f(); auto t1 = std::chrono::steady_clock::now(); double ms =
    std::chrono::duration<double, std::milli>(t1 - t0).count(); if (ms < b) b = ms; } return b; }
int main()
{
    const int N = 100000; volatile std::size_t sink = 0;
    double t1 = best([&] { Text s; Text p("x"); for (int i = 0; i < N; ++i) s = s + p; sink += s.size(); });
    double t2 = best([&] { Text s; Text p("x"); for (int i = 0; i < N; ++i) s += p;      sink += s.size(); });
    double t3 = best([&] { std::string s; std::string p("x"); for (int i = 0; i < N; ++i) s = s + p; sink +=
        s.size(); });
    double t4 = best([&] { std::string s; std::string p("x"); for (int i = 0; i < N; ++i) s += p;      sink +=
        s.size(); });
    std::printf("Text     s = s + p : %9.2f ms\n", t1);
    std::printf("Text     s += p    : %9.2f ms\n", t2);
    std::printf("std::string s = s + p : %9.2f ms\n", t3);
    std::printf("std::string s += p    : %9.2f ms\n", t4);
}
```

Output: Q6/bench_O2.txt

```
$ g++ -std=c++17 -O2 q6_bench.cpp && ./a.out
Text         s = s + p :    233.44 ms
Text         s += p    :    120.13 ms
```

```
std::string s = s + p :     120.20 ms
std::string s += p    :       0.21 ms
```

Test (100,000 appends, -O2)	Time	Why
Text, s = s + p	233 ms	copy of s into a by-value parameter, then += reallocates: two O(n) copies per step; O(N^2) in total
Text, s += p	120 ms	one O(n) copy per step (allocates exactly the new size every time); O(N^2) in total
std::string, s = s + p	120 ms	builds a new string every time: O(n) per step, O(N^2)
std::string, s += p	0.21 ms	amortised O(1) per append because capacity grows geometrically; **about 570x faster**

Text matches std::string when both build a new string each time, but std::string wins by a huge factor where it has spare capacity. Text re-allocates the exact size on every append.

Self-assignment: tested with `s = s` and `s = s = s` (output above). The guard `if (this == &o) return *this;` matters because without it the assignment would `delete[]` the array and then copy from the freed block (undefined behaviour).

4. Conclusion

Writing `Text` showed how ownership, deep copying, the Rule of Three, exactness of allocation sizes (the off-by-one) and operator return types fit together, and it made the cost of every temporary visible (29 characters copied, 2 allocations for three operands). In production code `std::string` should be used: it is tested, has move semantics, small-string optimisation and capacity management. I would still write such a class when the memory layout is constrained (fixed arena, shared memory, an embedded system without a heap), when interoperating with a C API that requires specific ownership rules, or as a teaching exercise.

5. Alternative: std::string, or + implemented with += on a by-value copy

Source: Q6/q6_versions.cpp

```
// Alternative: operator+(Text a, const Text& b){ a += b; return a; }  vs  operator+(const Text&, const Text&)
// Count chars copied and allocations for a + b + c with n1=1000, n2=2000, n3=3000
#include <cstring>
#include <cstdio>
#include <string>
static long allocs = 0, chars = 0;
class Text {
    char* d; std::size_t n;
public:
    Text(const char* s = "") : n(std::strlen(s)) { d = new char[n + 1]; ++allocs; chars += (long)n; std::memcpy(d, s,
        n + 1); }
    Text(const Text& o) : n(o.n) { d = new char[n + 1]; ++allocs; chars += (long)n; std::memcpy(d, o.d, n + 1); }
    ~Text() { delete[] d; }
    Text& operator+=(const Text& o) { char* f = new char[n + o.n + 1]; ++allocs; chars += (long)(n + o.n);
        std::memcpy(f, d, n); std::memcpy(f + n, o.d, o.n + 1); delete[] d; d = f; n += o.n; return *this; }
    std::size_t size() const { return n; }
    friend Text V1(const Text& a, const Text& b) { Text r; r.n = a.n + b.n; delete[] r.d; r.d = new char[r.n + 1];
        ++allocs; chars += (long)r.n; std::memcpy(r.d, a.d, a.n); std::memcpy(r.d + a.n, b.d, b.n + 1); return r; }
    friend Text V2(Text a, const Text& b) { a += b; return a; }
};
int main()
{
    std::string s1(1000, 'a'), s2(2000, 'b'), s3(3000, 'c');
    Text a(s1.c_str()), b(s2.c_str()), c(s3.c_str());
    allocs = chars = 0; { Text r = V1(V1(a, b), c); std::printf("V1 const& builds result : allocations=%ld chars
        copied=%ld (len %zu)\n", allocs, chars, r.size()); }
    allocs = chars = 0; { Text r = V2(V2(a, b), c); std::printf("V2 by-value + '+='       : allocations=%ld chars
        copied=%ld (len %zu)\n", allocs, chars, r.size()); }
}
```

a + b + c with n1 = 1000, n2 = 2000, n3 = 3000

```
$ g++ -std=c++17 -O0 q6_versions.cpp && ./a.out
V1 const& builds result : allocations=4 chars copied=9000 (len 6000)
V2 by-value + '+='      : allocations=5 chars copied=19000 (len 6000)
```

Version	Characters copied for a+b+c	Allocations	Temporary space
V1: operator+(const Text&, const Text&) builds result	(n1+n2) + (n1+n2+n3) = 9,000	2 results (+2 empty temporaries in this instrumented class)	one intermediate of n1+n2
V2: operator+(Text a, const Text& b){ a += b; return a; }, no move constructor	19,000 = [n1 + (n1+n2) + return copy (n1+n2)] + [(n1+n2+n3) + return copy (n1+n2+n3)]	5	intermediate copies of n1, n1+n2, n1+n2, n1+n2+n3 live temporarily
std::string (move semantics, capacity)	about n1 + n2 + n3 (appends to the moved-from temporary)	1-2	one intermediate

V2 is the idiom recommended for classes *with* a move constructor, because the by-value parameter can steal an rvalue and the return is moved. My Text has no move constructor, so the returned parameter had to be *copied*, and V2 copied about twice as many characters as V1 (19,000 vs 9,000). Time is proportional to characters copied, i.e. O(n1 + n2 + n3) in all versions, but with constants of 1x (std::string), 2x (V1) and about 3x (V2 without move). Adding a move constructor (the Rule of Five) makes V2 efficient.

Q7 - Design a Complex-Number Class from First Principles

Stage 1 - Interpret the problem

- **Data:** an impedance Z = R + jX is two doubles: the real part (resistance R) and the imaginary part (reactance X). Resistor: R + j0. Inductor: 0 + j(wL). Capacitor: 0 - j/(wC), with w = 2 pi f.
- **Operations needed:** addition (series), division and multiplication (parallel: Z1 x Z2 / (Z1 + Z2)), subtraction (for completeness and differences), magnitude |Z| = sqrt(R^2 + X^2), equality test (for verification), printing in rectangular form.
- **Constructors needed:** default `Complex()` = 0 + j0; from a real part `Complex(r)` (so a plain resistance converts and `2 + z` works); from real and imaginary parts `Complex(r, i)`. All three are provided by one constructor with default arguments.

Requirements as measurable statements:

ID	Requirement	How it is measured
R1	Series impedance of R, L, C equals R + j(wL - 1/(wC))	Agrees with a hand calculation to 4 decimal places
R2	Parallel impedance uses (Z1 x Z2)/(Z1 + Z2) and is correct for 3 components	Compared with the admittance method by hand; same to 4 decimals
R3	Output in rectangular form plus magnitude	Printed as "a + jb" / "a - jb" with 4 decimals, magnitude 4 decimals
R4	Division by a zero impedance never produces nan/inf or a crash	Throws std::domain_error in 3 test cases
R5	Each operator is O(1) time and the object is 16 bytes	Two doubles, no heap use

Stage 2 - Design

Operator	Form	Formula (a = a1 + j a2, b = b1 + j b2)	Why this form
`+= -=`	member	a1 +- b1 , a2 +- b2	modifies *this, returns reference
`*=`	member	(a1 b1 - a2 b2) + j(a1 b2 + a2 b1)	uses old a1 before overwrite
`/=`	member	d = b1^2 + b2^2 ; re = (a1 b1 + a2 b2)/d ; im = (a2 b1 - a1 b2)/d ; throws if d = 0	central place for the zero check
unary `-`	member	-a1 - j a2	one operand
`+ - * /`	friend (non-member)	copy of left operand, then `op=`	so both `2 + z` and `z + 2` work
`== !=`	friend	|a1 - b1| and |a2 - b2| within 1e-9 (relative)	floating-point numbers are never compared exactly

Operator	Form	Formula (a = a1 + j a2, b = b1 + j b2)	Why this form
<<	friend	prints "re + jim" or "re - j|im|", 4 decimals	left operand is std::ostream

Helper functions `resistor(R)`, `inductor(w, L)`, `capacitor(w, C)` create the three component impedances, and `parallel(a, b)` implements (a x b)/(a + b). `magnitude()` uses `std::hypot` to avoid overflow in the intermediate squares.

Stage 3 - Implement and verify

Source: Q7/q7_complex.cpp

```
// Q7 - Complex-number class for an AC impedance calculator
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>

class Complex {
    double re, im;
public:
    Complex(double r = 0.0, double i = 0.0) : re(r), im(i) {}           // default, from real part, from real+imag
    double real() const { return re; }
    double imag() const { return im; }
    double magnitude() const { return std::hypot(re, im); }             // |Z| = sqrt(re^2 + im^2), overflow-safe
    double phaseDegrees() const { return std::atan2(im, re) * 180.0 / 3.14159265358979323846; }

    Complex operator-() const { return Complex(-re, -im); }
    Complex& operator+=(const Complex& o) { re += o.re; im += o.im; return *this; }
    Complex& operator-=(const Complex& o) { re -= o.re; im -= o.im; return *this; }
    Complex& operator*=(const Complex& o) { double r = re * o.re - im * o.im; im = re * o.im + im * o.re; re = r;
        return *this; }
    Complex& operator/=(const Complex& o)
    {
        double d = o.re * o.re + o.im * o.im;                            // |o|^2
        if (d == 0.0) throw std::domain_error("division by a zero impedance");
        double r = (re * o.re + im * o.im) / d;
        im = (im * o.re - re * o.im) / d; re = r;
        return *this;
    }
    // non-members so that  2.0 + z  and  z + 2.0  both work (the double converts to Complex)
    friend Complex operator+(Complex a, const Complex& b) { return a += b; }
    friend Complex operator-(Complex a, const Complex& b) { return a -= b; }
    friend Complex operator*(Complex a, const Complex& b) { return a *= b; }
    friend Complex operator/(Complex a, const Complex& b) { return a /= b; }
    friend bool operator==(const Complex& a, const Complex& b)          // tolerance: floating point is never exact
    {
        const double eps = 1e-9;
        return std::fabs(a.re - b.re) <= eps * (1 + std::fabs(b.re)) && std::fabs(a.im - b.im) <= eps * (1 +
            std::fabs(b.im));
    }
    friend bool operator!=(const Complex& a, const Complex& b) { return !(a == b); }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z)
    {
        std::ios::fmtflags f = os.flags(); std::streamsize p = os.precision();
        os << std::fixed << std::setprecision(4) << z.re << (z.im < 0 ? " - j" : " + j") << std::fabs(z.im);
        os.flags(f); os.precision(p); return os;
    }
};

// component impedances
Complex resistor(double R)                { return Complex(R, 0); }
Complex inductor(double w, double L)      { return Complex(0, w * L); }
Complex capacitor(double w, double C)     { return Complex(0, -1.0 / (w * C)); }
Complex parallel(const Complex& a, const Complex& b) { return (a * b) / (a + b); }     // may throw

int main()
{
    std::cout << std::fixed << std::setprecision(4);
    std::cout << "=== Circuit A: R=100 ohm, L=0.1 H, C=10 uF, w=1000 rad/s (resonance) ===\n";
    {
        double w = 1000, R = 100, L = 0.1, C = 10e-6;
        Complex zr = resistor(R), zl = inductor(w, L), zc = capacitor(w, C);
        std::cout << "ZR = " << zr << "\nZL = " << zl << "\nZC = " << zc << '\n';
        Complex zs = zr + zl + zc;
        std::cout << "Series   Z = " << zs << "   |Z| = " << zs.magnitude() << '\n';
        Complex zp = parallel(parallel(zr, zl), zc);
        std::cout << "Parallel Z = " << zp << "   |Z| = " << zp.magnitude() << "   (R||L = " << parallel(zr, zl) <<
            ")\n";
    }
    std::cout << "=== Circuit B: same parts at 50 Hz (w = 314.159 rad/s) ===\n";
    {
        double w = 2 * 3.14159265358979 * 50, R = 100, L = 0.1, C = 10e-6;
        Complex zr = resistor(R), zl = inductor(w, L), zc = capacitor(w, C);
        Complex zs = zr + zl + zc, zp = parallel(parallel(zr, zl), zc);
```

```
        std::cout << "ZL = " << zl << "   ZC = " << zc << '\n';
        std::cout << "Series   Z = " << zs << "   |Z| = " << zs.magnitude() << "   phase = " << zs.phaseDegrees() <<
            " deg\n";
        std::cout << "Parallel Z = " << zp << "   |Z| = " << zp.magnitude() << "   phase = " << zp.phaseDegrees() <<
            " deg\n";
    }
    std::cout << "=== operator tests ===\n";
    Complex a(3, 4), b(1, -2);
    std::cout << "(3+j4)+(1-j2) = " << a + b << "\n(3+j4)-(1-j2) = " << a - b
              << "\n(3+j4)*(1-j2) = " << a * b << "   (hand: 11 - j2)\n(3+j4)/(1-j2) = " << a / b
              << "   (hand: -1 + j2)\n|3+j4| = " << a.magnitude() << "\n2 + (3+j4) = " << 2 + a << "   (3+j4) * 2 = "
                  << a * 2 << '\n';
    std::cout << std::boolalpha << "(3+j4)*(3-j4) == 25 ? " << (a * Complex(3, -4) == Complex(25)) << '\n';
    std::cout << "=== division by zero impedance ===\n";
    try { Complex z = a / Complex(0, 0); std::cout << z << '\n'; }
    catch (const std::domain_error& e) { std::cout << "caught: " << e.what() << '\n'; }
    try { std::cout << parallel(Complex(0, 0), Complex(0, 0)) << '\n'; }                  // two shorts in parallel
    catch (const std::domain_error& e) { std::cout << "caught (parallel of two shorts): " << e.what() << '\n'; }
    try { Complex zl = inductor(1000, 0.1), zc = Complex(0, -100); std::cout << parallel(zl, zc) << '\n'; }    //
        ideal L||C at resonance
    catch (const std::domain_error& e) { std::cout << "caught (ideal L || C at resonance, Z1+Z2 = 0): " << e.what()
        << '\n'; }
}
```

Output: Q7/out_complex.txt

```
$ g++ -std=c++17 -Wall -Wextra q7_complex.cpp && ./a.out
=== Circuit A: R=100 ohm, L=0.1 H, C=10 uF, w=1000 rad/s (resonance) ===
ZR = 100.0000 + j0.0000
ZL = 0.0000 + j100.0000
ZC = 0.0000 - j100.0000
Series   Z = 100.0000 + j0.0000   |Z| = 100.0000
Parallel Z = 100.0000 + j0.0000   |Z| = 100.0000   (R||L = 50.0000 + j50.0000)
=== Circuit B: same parts at 50 Hz (w = 314.159 rad/s) ===
ZL = 0.0000 + j31.4159   ZC = 0.0000 - j318.3099
Series   Z = 100.0000 - j286.8940   |Z| = 303.8226   phase = -70.7834 deg
Parallel Z = 10.8333 + j31.0800   |Z| = 32.9139   phase = 70.7834 deg
=== operator tests ===
(3+j4)+(1-j2) = 4.0000 + j2.0000
(3+j4)-(1-j2) = 2.0000 + j6.0000
(3+j4)*(1-j2) = 11.0000 - j2.0000   (hand: 11 - j2)
(3+j4)/(1-j2) = -1.0000 + j2.0000   (hand: -1 + j2)
|3+j4| = 5.0000
2 + (3+j4) = 5.0000 + j4.0000   (3+j4) * 2 = 6.0000 + j8.0000
(3+j4)*(3-j4) == 25 ? true
=== division by zero impedance ===
caught: division by a zero impedance
caught (parallel of two shorts): division by a zero impedance
caught (ideal L || C at resonance, Z1+Z2 = 0): division by a zero impedance
```

Hand verification.

- **Circuit A** (w = 1000, R = 100, L = 0.1 H, C = 10 uF): ZL = j(1000 x 0.1) = j100; ZC = -j/(1000 x 10e-6) = -j100. *Series:* 100 + j(100 - 100) = **100 + j0**, |Z| = 100 (resonance). *Parallel:* R||L = (100 x j100)/(100 + j100) = j100/(1 + j) = j100(1 - j)/2 = 50 + j50. Then (50 + j50) || (-j100) = (50 + j50)(-j100)/(50 - j50) = (5000 - j5000)/(50 - j50) = **100 + j0**. Program: 100.0000 + j0.0000 and intermediate 50 + j50. Matches.
- **Circuit B** (50 Hz, w = 314.159): ZL = j31.4159, ZC = -j318.3099. *Series:* 100 + j(31.4159 - 318.3099) = **100 - j286.894**, |Z| = sqrt(100^2 + 286.894^2) = 303.82; phase = atan(-2.86894) = -70.78 deg. *Parallel by admittances:* Y = 1/100 + 1/(j31.4159) + 1/(-j318.3099) = 0.01 - j0.031831 + j0.003142 = 0.01 - j0.028690; Z = 1/Y = (0.01 + j0.028690)/(0.01^2 + 0.028690^2) = **10.833 + j31.080**, |Z| = 32.91. Program: 10.8333 + j31.0800, |Z| = 32.9139. Matches.
- **Operator checks:** (3+j4)(1-j2) = 3 - j6 + j4 - j^2 8 = 11 - j2; (3+j4)/(1-j2) = (3+j4)(1+j2)/5 = (-5 + j10)/5 = -1 + j2; |3+j4| = 5. All as printed.

Division by a zero impedance: `a / Complex(0,0)`, `parallel(0, 0)` (two shorts) and the ideal L || C at resonance (Z1 + Z2 = 0) each raise `std::domain_error`, caught and reported; no nan/inf is produced.

Stage 4 - Defend the design

Source: Q7/q7_alt.cpp

```
// Same calculations with std::complex<double>; division by zero is NOT reported
#include <complex>
#include <iostream>
#include <iomanip>
using Z = std::complex<double>;
Z parallel(Z a, Z b) { return (a * b) / (a + b); }
int main()
{
    std::cout << std::fixed << std::setprecision(4);
```

```
    double w = 1000, R = 100, L = 0.1, C = 10e-6;
    Z zr(R, 0), zl(0, w * L), zc(0, -1 / (w * C));
    std::cout << "series   : " << zr + zl + zc << "  |Z| = " << std::abs(zr + zl + zc) << '\n';
    std::cout << "parallel : " << parallel(parallel(zr, zl), zc) << '\n';
    Z a(3, 4), b(1, -2);
    std::cout << "(3+4j)*(1-2j) = " << a * b << "   (3+4j)/(1-2j) = " << a / b << '\n';
    std::cout << "(3+4j)/(0+0j)           = " << a / Z(0, 0) << "   <- no exception, silent nan/inf\n";
    std::cout << "parallel(0, 0)          = " << parallel(Z(0, 0), Z(0, 0)) << "   <- nan\n";
    std::cout << "parallel(+100j, -100j) = " << parallel(Z(0, 100), Z(0, -100)) << "   <- nan/inf at resonance\n";
}
```

Same calculations with std::complex<double>

```
$ g++ -std=c++17 q7_alt.cpp && ./a.out
series   : (100.0000,0.0000)  |Z| = 100.0000
parallel : (100.0000,0.0000)
(3+4j)*(1-2j) = (11.0000,-2.0000)    (3+4j)/(1-2j) = (-1.0000,2.0000)
(3+4j)/(0+0j)           = (inf,inf)    <- no exception, silent nan/inf
parallel(0, 0)          = (-nan,-nan)    <- nan
parallel(+100j, -100j) = (inf,-nan)    <- nan/inf at resonance
```

Aspect	My Complex	std::complex<double>
Time per operation	O(1)	O(1)
Space	2 doubles = 16 bytes	2 doubles = 16 bytes
Results for the test circuits	identical to 4 decimals	identical
Division by zero	throws domain_error, caught by the calculator	silent: (inf,inf), -nan; the caller must test for it
Accuracy for extreme magnitudes	naive formula: |b|^2 can overflow (about 1e154) or underflow	library routine with scaling, more robust
Extra functions	only what the lab needs	polar form, exp, sqrt, trigonometric functions

- **Why each constructor:** the default constructor allows arrays and uninitialised variables to be zero; the one-argument form lets a pure resistance convert (R + j0) and makes `2 + z` work; the two-argument form builds inductors and capacitors.
- **Why each operator:** `+` for series; `*` and `/` for parallel; `-` for differences and the unary negative; `==` for verification against hand results (with tolerance); `<<` for the printed rectangular form.
- **One limitation:** the class cannot represent infinite impedance (an ideal open circuit, or L || C exactly at resonance) or an exact short; these cases are reported by an exception rather than as a value. A second consequence: `==` with tolerance is not transitive, so it must not be used for sorting or as a map key.

Alternative: use std::complex<double> instead

The solution would shrink to `using Z = std::complex<double>;` plus the three helper functions and a `parallel()` that first checks `a + b == Z(0)` before dividing, because std::complex does not report division by zero. Time per operation is O(1) in both designs and the space is two doubles in both. For accuracy, std::complex is at least as good (scaled division) and for safety my class is better out of the box because the zero check is built into `/`. In production I would use std::complex with a small checking wrapper; writing my own class was valuable for understanding member/friend choices, operator return types and tolerance in `==`.

Appendix - Repository layout, README and commit plan

```
DSA01-192421026-Assignment3/
  README.md
  Q1/  q1_book.cpp q1_nodefault.cpp q1_reorder.cpp q1_explicit.cpp q1_alt.cpp    run.sh  out_*.txt err_*.txt
  Q2/  q2_complete.cpp q2_shallow.cpp q2_counts.cpp q2_bench.cpp                 run.sh  asan_*.txt bench_O2.txt
  Q3/  q3_auditorium.cpp q3_leak.cpp q3_predict.cpp q3_alt.cpp                   run.sh  asan_*.txt out_*.txt
  Q4/  q4_complete.cpp q4_money.cpp q4_readability.cpp q4_double.cpp q4_friend_vs_member.cpp  run.sh
  Q5/  q5_playlist.cpp q5_predict.cpp q5_bench.cpp                               run.sh  bench.txt
  Q6/  q6_text.cpp q6_bug.cpp q6_bench.cpp q6_versions.cpp                       run.sh  asan_bug.txt bench_O2.txt
  Q7/  q7_complex.cpp q7_alt.cpp                                                 run.sh  out_*.txt
```

README.md contains: name, register number, compiler and version (g++ 13.3.0), compile command (`g++ -std=c++17 -Wall -Wextra file.cpp -o file`; sanitizer runs add `-fsanitize=address -g`; benchmarks add `-O0` / `-O2`),

how to run each program (`bash run.sh` in each folder reproduces every output file), and a one-line result for each task.

Commit plan (at least one commit per task, on different days): "Q1: constructors and delegation", "Q2: deep copy and ASan", "Q3: dynamic constructor/destructor", "Q4: Money operators", "Q5: ++ and [] operators", "Q6: Text class", "Q7: Complex class", "Add report and README".

Viva preparation: (a) why the by-value copy and the delegated constructor are different from assignment in the body; (b) why the shallow copy crashes and what the Rule of Three says; (c) the order of construction and destruction in the Q3 output; (d) why `<<` is a non-member and `+=` returns a reference; (e) prefix vs postfix and the dummy int; (f) why operator= is a member; (g) what happens in std::complex when you divide by zero.